



Année Universitaire : 2023 / 2024

Génie informatique

Option Génie logiciel

Mémoire de Projet de Fin d'Etudes

Intitulé :

**Détection de Fruits Datés à l'aide d'Algorithmes
d'Apprentissage en Profondeur
(En anglaise: Date Fruit Detection using Deep
Learning Algorithms)**

Préparé par :

- Bouhani ELMustafa
- Boudaoud Aissa

Soutenu le Juillet 2024 devant le Jury :

- Pr. Mohamed OUHDA (Encadrant).

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce rapport de fin d'étude intitulé "découvrir les fruits des dattes à l'aide d'algorithmes d'apprentissage en profondeur".

À nos parents

Nous exprimons notre gratitude à nos familles pour leur amour et leur soutien constants. Leur confiance en NOUS a été une source inestimable de motivation.

À notre encadrant

Nous adressons nos sincères remerciements à notre encadrant, Mohamed OUHDA, pour son encadrement exceptionnel et ses conseils avisés tout au long de ce projet. Ses vastes connaissances et sa disponibilité ont été essentielles à notre progression. Nous avons particulièrement apprécié ses suggestions lors de nos réunions, qui ont toujours été très constructives.

À nos professeurs

Nous tenons également à remercier le personnel enseignant de l'école supérieure de technologie 'Moulay Slimane' pour son soutien et ses précieux enseignements. Chaque cours a enrichi notre compréhension et notre compétence au cours de nos études universitaires.

Enfin, nous remercions toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce travail, y compris celles qui ont participé aux tests et à la validation des algorithmes.

Merci à tous.

Résumé

Dans ce travail, nous proposons un cadre de vision industrielle efficace pour les robots de récolte des dattes. Le cadre se compose de trois modèles de classification utilisés pour classer les images de fruits datés en temps réel en fonction de leur type, de leur maturité et de leur décision de récolte. Dans les modèles de classification, les réseaux neuronaux convolutifs profonds sont utilisés avec l'apprentissage par transfert et le réglage fin sur des modèles pré-entraînés. Pour construire un système de vision robuste, nous créons un riche ensemble de données d'images de grappes de dattes dans un verger qui se compose de plus de 9092 images de cinq types de dattes à différents stades de prématurité et de maturité. L'ensemble de données présente un grand degré de variations qui reflète les défis de l'environnement des vergers de dattiers, notamment les variations d'angles, d'écailles, de conditions d'éclairage et les régimes de dattes recouverts de sacs. Les modèles de classification des dattiers proposés atteignent des précisions de 90,01 %, 90,25 % et 90,59 % avec des temps de classification de 20,6, 20,7 et 35,9 ms pour les tâches de classification du type, de la maturité et de la décision de récolte, respectivement.

Table des Tableaux :

Tableau 1 : Des différentes versions et évolutions de YOLO.

Tableau 2 : Les versions du modèle YOLOv8 pour la détection.

Tableau 3 : Les versions du modèle YOLOv8 pour la classification.

Tableau 4 : Répartition par Stade de Maturité

Tableau 5 : Utilisation du Pourcentage de Maturité pour la Récolte.

Tableau 6 : Nombre d'Images par Variété de Dattes.

Table des figures

Figure 1 : Une image des palmiers.

Figure 2 : image a plusieurs pixels.

Figure 3 : image a niveau de gris.

Figure 4 : les couleurs de RGB.

Figure 5 : Image couleur.

Figure 6 : Image binaire.

Figure 7 : La différence entre l'image matricielle et l'image vectorielle.

Figure 8 : Une image avec différent bruit.

Figure 9 : Image avec son histogramme.

Figure 10 : Une image et la détection des dattes.

Figure 11 : La relation entre l'IA, le ML et le Deep Learning.

Figure 12 : Réseau de neurones convolutifs.

Figure 13 : Architecture d'un CNN.

Figure 14 : Split.

Figure 15 : résume les différents modèles du deep Learning.

Figure 16 : résume les différents modèles du deep Learning.

Figure 17 : Les domaines de la vision par ordinateur.

Figure 18 : Schéma représentant l'approche de vision par ordinateur.

Figure 19 : la représentation de k-means avec $k=4$.

Figure 20 : Illustration de la performance de YOLO.

Figure 21 : l'organisation de dataset dans le fichier data.yaml.

Figure 22 : Une image annoter avec Roboflow.

Figure 23 : Annotation d'image précédente.

Figure 24 : Code d'importation du modèle yolov8.

Figure 25 : Code d'entraînement du modèle yolov8.

Figure 26 : l'organisation de dataset pour la classification selon le type et selon la maturité.

Figure 27 : Interface de Google Colab.

Figure 28 : La courbe précision-rappel.

Figure 29 : La matrice de confusion normalisée

Figure 30 : La matrice de confusion normalisée

Figure 31 : les performances du modèle de classification au cours de l'entraînement

Figure 32 : page d'accueil

Figure 33 : d'importation des images

Figure 34 : page des résultats.

Figure 35 : Historique des analyses des images précédentes.

Figure 36 : Résultats de classification par variété

Figure 37 : Résultat de la classification par maturité

Table des matières

Introduction générale.....	1
Chapitre 1 : Les notions de traitement d'image.....	3
.I Introduction :	3
II. Notion d'image :	3
II.1 Image numérique :	4
II.2 Type des images numérique :	4
II.2.1 Image matricielle :	4
II.2.2 Image vectorielle :	7
II.2.3 Caractéristiques d'une image numérique :	8
III. La détection et la classification des objets :	9
III.1 La détection des objets :	9
III.1.1 Définition :	9
III.1.2 Les grandes étapes de la détection des objets :	10
III.2 La classification des images :	11
III.2.1 Définition :	11
III.2.2 Les grandes étapes de la classification des images :	11
IV. Conclusion :	11
Chapitre 2 : La solution proposé – YOLOv8.....	12
I. Introduction :	12
II. Intelligence Artificielle :	12
II.1 Définition :	12
II.2 Relation entre l'intelligence artificielle, le ML et le Deep Learning :	13
III. Machine Learning (ML) :	14
IV. Deep Learning (DL) :	14
IV.1 Définition :	14
IV.2 Les réseaux de neurones convolutifs :	15
IV.2.1 Architecture d'un réseau de neurones convolutifs :	15
IV.2.2 Caractéristiques et avantages des CNN :	15
I.1.1.1.1. Convolution d'image :	17
IV.3 Révolution d'apprentissage profond :	24

IV.3.1 L'évolution des CPU et des GPU :.....	24
IV.3.2 Une Activation du GPU :.....	24
V. Vision par ordinateur :.....	25
VI. K- means :.....	27
VI.1 Définition et principes :.....	27
VI.2 Avantages de l'algorithme :	29
VI.3 Cet algorithme à plusieurs inconvénients :.....	29
VII. Environnement de développement :.....	30
VII.1 Environnement matériel (hardware) :	30
VII.2 Environnement immatériel (software) :	30
VIII. YOLO (You Only Look Once) :.....	30
VIII.1 Définition (Yolo, c'est quoi?) :.....	30
VIII.2 Les versions de YOLO :.....	30
VIII.2.1 YOLO V1 :.....	31
VIII.2.2 YOLO V2 et V3:	31
VIII.2.3 YOLO V4 et V5:	31
VIII.2.4 YOLO V6 à V8:	32
VIII.2.5 YOLO V9:	32
VIII.3 Fonctionnement de YOLO:.....	33
VIII.3.1 Prendre une photo :.....	33
VIII.3.2 Diviser l'image :	34
VIII.3.3 Chercher des indices :.....	34
VIII.3.4 Faire des prédictions :.....	34
VIII.3.5 Éliminer les excédents :	34
VIII.3.6 Montrer ce qu'il a trouvé :.....	34
VIII.4 Evolution de la détection d'objets : de YOLO 1 à YOLO 9 :.....	35
VIII.5 La version utilisée – YOLOv8 :.....	35
VIII.5.1 YOLOv8 pour la Détection d'Objets :.....	36
VIII.5.2 YOLOv8 pour la Classification d'Images :.....	39
IX. Conclusion :.....	41
Chapitre 3 : Description de dataset.....	42
I. Introduction :.....	42

II. La description des données :.....	42
II.1 Conception Expérimentale, Matériaux et Méthodes	43
II.1.1 Caractéristiques principales :.....	43
II.1.2 Valeur des Données :	43
II.1.3 Description Détaillée des Données	44
II.2 Limitations :	46
II.2.1 Déclaration d'Éthique	46
II.2.2 Disponibilité des Données.....	46
III. Les outils utilisés :	46
 III.1 Langage de programmation Python :python™	46
III.2 Google colab :	47
III.3 Google Drive :	48
IV. Conclusion	48
Chapitre 4 : L'interface web.....	49
I. Introduction :	49
II. Les résultats obtenus avec YOLOv8 :	49
II.1 La courbe précision-rappel :	49
II.2 La matrice de confusion normalisée :	50
II.3 La matrice de confusion normalisée :	51
II.4 Des graphiques de performance du modèle :	53
III. L'interface graphique :	54
IV. Conclusion :	58
Conclusion générale	59
Bibliographie	61

Introduction générale

La région de Drâa-Tafilalet est l'une des principales zones de production de dattes au Maroc, couvrant plus de 48 453 hectares, y compris les nouvelles zones d'extension sur l'axe Meski-Boudnib. Cette région produit environ 94 000 tonnes de dattes par an, soit 85% de la production nationale pour la période 2010-2018, et emploie plus de 100 000 producteurs regroupés en 21 groupements d'intérêt économique.

Face aux défis posés par la récolte manuelle, ce projet vise à améliorer les processus de récolte grâce à la détection et à la classification des images. En tant qu'informaticiens, nous avons la capacité de contribuer de manière significative en développant des outils avancés pour le traitement et la manipulation des images. La détection précise et la classification fiable des dattes optimisent la récolte en automatisant l'identification de la maturité et de la variété des fruits, ce qui réduit le temps et les efforts nécessaires pour les trier manuellement.

L'objectif principal de ce projet est de développer une interface graphique intégrant un outil de détection et de classification des dattes en utilisant YOLOv8 (You Only Look Once version 8). YOLOv8 est un modèle de détection d'objets de pointe qui permet d'entraîner des modèles robustes capables de détecter et de classer les fruits de dattes selon leur maturité, en diverses conditions d'éclairage et d'angle.

Ce travail se décompose en plusieurs étapes clés :

Étude Théorique : Compréhension des bases de la détection et la classification d'images et des techniques de vision par ordinateur.

Implémentation avec YOLOv8 : Utilisation de YOLOv8 pour détecter et classifier les dattes. YOLOv8 permet de traiter efficacement les images pour reconnaître les fruits de dattes et déterminer leur stade de maturité.

Analyse et Modélisation : Développement de l'application incluant une interface utilisateur pour le traitement des images et la visualisation des résultats.

Grâce à ce projet, nous espérons automatiser le processus de récolte des dattes, augmentant ainsi la productivité et la précision, tout en réduisant les coûts de main-d'œuvre dans la région de Drâa-Tafilalet.



Figure 1 : Une image des palmiers.

Chapitre 1 : Les notions de traitement d'image

I. Introduction :

Les humains communiquent par divers moyens, dont la parole et l'image sont d'une importance cruciale. L'image en particulier est un puissant moyen de communication, qui permet à chacun de déchiffrer et d'extraire des informations selon sa propre perspective.

Ce chapitre explore les concepts de base liés au traitement d'image, ainsi que les principales caractéristiques nécessaires à sa détection et à sa classification. L'objectif est de rendre ces processus plus faciles, plus efficaces et plus réalisables, tout en permettant une analyse précise et pratique des données visuelles.

II. Notion d'image :

Une image est une représentation visuelle de quelque chose, que ce soit une personne, un objet ou une scène. Cette représentation peut être réalisée par divers moyens tels que la peinture, la sculpture, le dessin, la photographie ou le film.

II.1 Image numérique :

Une image numérique est une représentation d'une scène visuelle ou d'un objet stocké sous la forme d'une collection de données binaires. Il est composé de pixels, qui sont les plus petites unités d'une image, chaque pixel ayant une valeur de couleur spécifique. Les images numériques peuvent être créées par des appareils tels que des appareils photo, des scanners ou générées par infographie. Ils sont couramment utilisés dans diverses applications telles que la photographie, l'imagerie médicale et la vision par ordinateur. La qualité et la résolution d'une image numérique dépendent du nombre de pixels et de la profondeur de représentation des couleurs.

II.2 Type des images numérique :

Les images numériques peuvent être classées en deux classes : les images matricielles et les images vectorielles.

II.2.1 Image matricielle :

C'est une image sous forme d'une matrice (représentée par une matrice) composées d'une grille de pixels individuels, où chaque pixel a une valeur de couleur spécifique. La qualité et les détails d'une image raster sont déterminés par sa résolution, mesurée en pixels par pouce (PPI) ou en points par pouce (DPI).

II.2.1.1 Pixel :

Le mot pixel vient de l'expression « picture element ». C'est l'unité indivisible permettant de coder l'information relative à la luminosité en certaine position dans l'image.

Un pixel est décrit par ses coordonnées dans l'image (i,j) , et sa valeur $I(i,j)$, représentant son niveau de gris ou sa couleur.

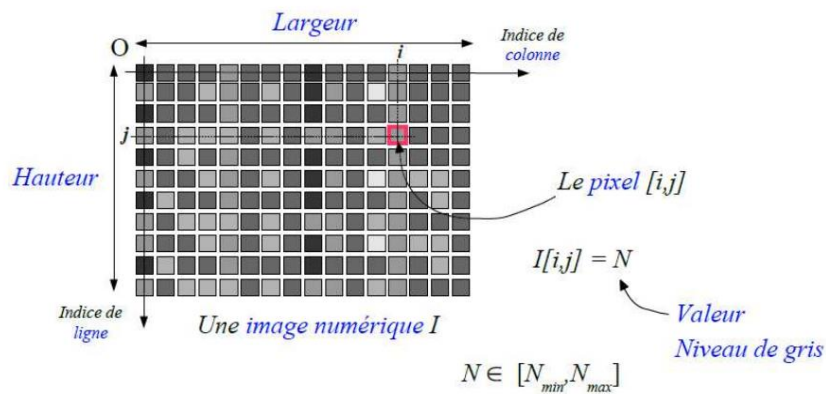


Figure 2 : image a plusieurs pixels.

II.2.1.2 Image a niveau de gris :

Une image à niveau de gris est une image matricielle où chaque pixel peut prendre une valeur entière de 0 à N, où N est la valeur maximale de niveau de gris.



Figure 3 : image a niveau de gris.

II.2.1.3 Image couleur :

Une image couleur est généralement représentée par trois matrices, chacune correspondant à un canal de couleur : rouge (Red), vert (Green) et bleu (Blue). Ces canaux, lorsqu'ils sont combinés, forment l'image complète.

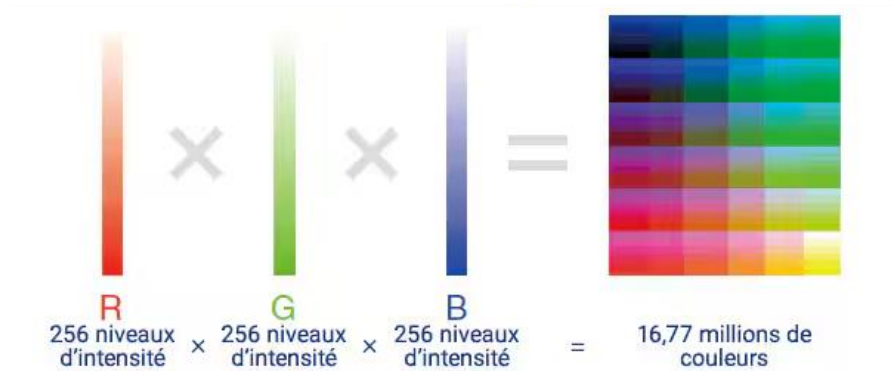


Figure 4 : les couleurs de RGB.



Figure 5 : Image couleur.

II.2.1.4 Image binaire :

Une image binaire est une image matricielle ($M \times N$) qui a uniquement deux valeurs possibles pour chaque pixel. Ces valeurs sont 0 et 1 ; 0 représentant le noir et 1 représentant le blanc.

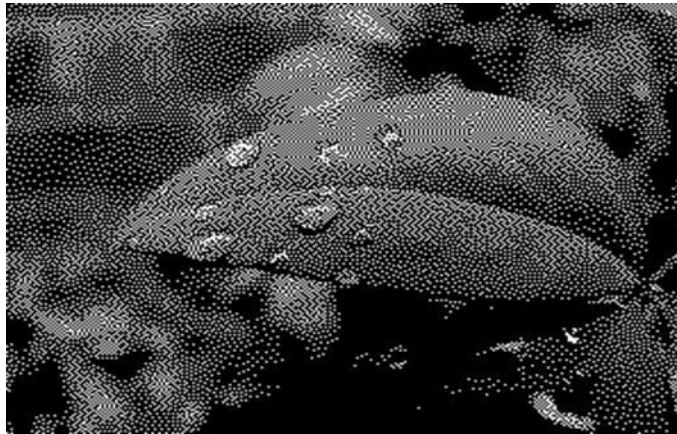


Figure 6 : Image binaire.

II.2.2 Image vectorielle :

Contrairement aux images matricielles formées de pixels individuels, les images vectorielles se basent sur des équations mathématiques et un réseau de points, de lignes, et de courbes pour représenter une image. Grâce à cette technique, les images vectorielles peuvent être agrandies ou réduites indéfiniment sans perdre en qualité, car les calculs mathématiques permettent d'ajuster l'image à la taille ou à la résolution désirée.



Figure 7 : La différence entre l'image matricielle et l'image vectorielle.

II.2.3 Caractéristiques d'une image numérique :

L'image est un ensemble structuré d'informations caractérisé par plusieurs paramètres, on cite quelques-uns :

II.2.3.1 Dimension :

Les dimensions d'une image numérique sont déterminées par sa résolution, c'est-à-dire le nombre de pixels en largeur et en hauteur. Par exemple, une photo a une résolution de 1920 * 1080 pixels par 1920 pixels et une largeur de 1080 pixels. La résolution fait référence au niveau de détail qu'une image peut afficher : soit une haute résolution ce qui signifie plus de pixels, et donc plus de détails. Ou basse résolution : signifie une image avec moins de pixels, ce qui signifie moins de détails.

II.2.3.2 Bruit :

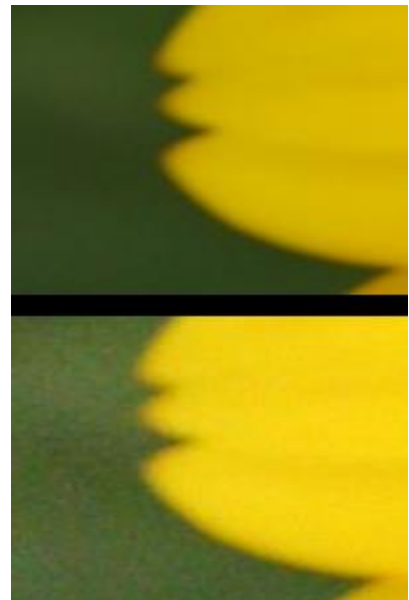
Un bruit (parasite) dans une image numérique fait référence aux variations indésirables des couleurs et de la luminosité qui n'existent pas dans la scène réelle. Le bruit peut être causé par divers facteurs, comme des capteurs de caméra de mauvaise qualité, des conditions de faible luminosité ou des compressions de fichiers.



Image A



Image B



ImageA– ImageB

Figure 8 : Une image avec différent bruit.

II.2.3.3 Histogramme :

L'histogramme des niveaux de gris ou des couleurs d'une image est une fonction qui donne la fréquence d'apparition de chaque niveau de gris (couleur) dans l'image. Il permet de donner un grand nombre d'information sur la majorité des niveaux de gris (couleur) dans le cas d'une image trop claire ou d'une image trop foncée.

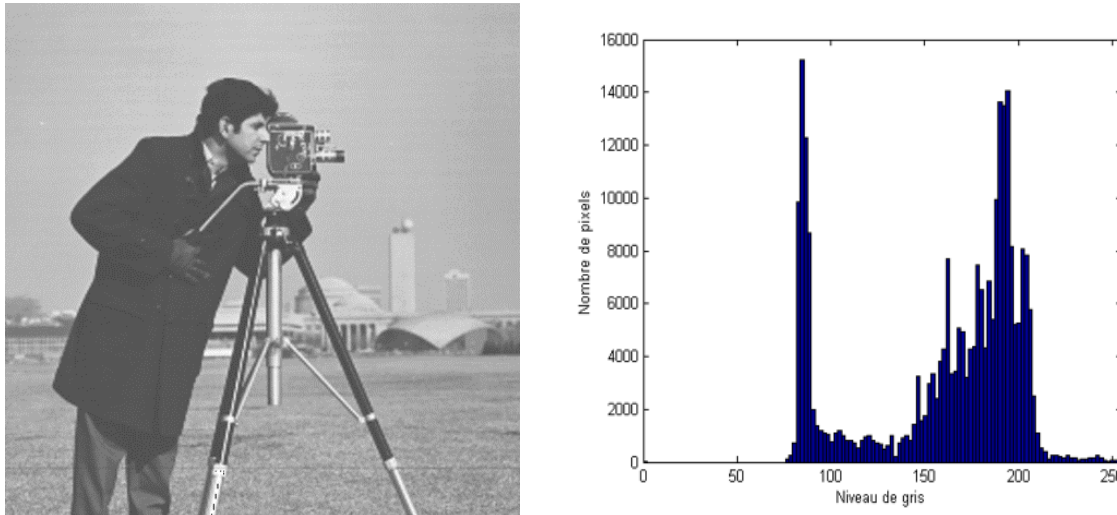


Figure 9 : Image avec son histogramme.

III. La détection et la classification des objets :

III.1 La détection des objets :

III.1.1 Définition :

La détection d'objets est une technologie de pointe en apprentissage automatique et en apprentissage profond qui permet aux ordinateurs d'identifier et de localiser avec précision les objets au sein des images ou des vidéos. Elle appartient à une branche de l'intelligence artificielle appelée "Computer Vision".

La détection d'objets est une tâche de vision par ordinateur dont l'objectif est de détecter et localiser des objets d'intérêt dans une image ou d'un flux vidéo. La tâche implique d'identifier la position et les limites des objets dans une image, ainsi que de classer les objets dans différentes catégories.

III.1.2 Les grandes étapes de la détection des objets :

En effet, le principe de la détection d'objets est le suivant : pour une image donnée, on recherche les régions de celle-ci qui pourraient contenir un objet puis pour chacune de ces régions découvertes, on l'extrait et on la classe à l'aide d'un modèle de classification d'image – par exemple –. Les régions de l'image d'origine ayant de bons résultats de détection sont conservés et les autres jetés. Ainsi, pour avoir une bonne méthode de détection d'objets, il est nécessaire d'avoir un algo solide de détection de régions et un bon algo de détection et classification d'images.

Alors, en se basant sur plusieurs données, soit une variété d'images et de données d'entraînement, l'algorithme IA détectera progressivement les motifs, les textures et les formes communes aux images de test utilisées pour chaque catégorie et apprendra à les reconnaître. Il pourra alors les identifier automatiquement dans n'importe quelle nouvelle image.



Figure 10 : Une image et la détection des dattes.

III.2 La classification des images :

III.2.1 Définition :

La classification des images est la plus simple des trois tâches et consiste à classer une image entière dans l'une des classes prédéfinies.

Le résultat d'un classificateur d'images est une étiquette de classe unique et un score de confiance. La classification d'images est utile lorsque tu as besoin de savoir uniquement à quelle classe appartient une image et que tu n'as pas besoin de savoir où se trouvent les objets de cette classe ou quelle est leur forme exacte (la localisation des objets est une tâche de la détection des objets).

III.2.2 Les grandes étapes de la classification des images :

La classification d'image implique de faire passer une image entière à travers un classificateur, généralement un réseau neuronal profond, pour obtenir une étiquette ou un tag correspondant. Les classificateurs analysent l'image complète mais ne fournissent pas d'informations sur l'emplacement spécifique de l'objet étiqueté à l'intérieur de l'image.

IV. Conclusion :

Nous avons introduit les notions de base qui servent de fondement à la compréhension de différents types d'images numériques ainsi que leurs caractéristiques, et aussi une définition générale de la détection et de la classification.

Chapitre 2 : La solution proposé – YOLOv8

I. Introduction :

Dans ce chapitre, nous allons explorer les concepts fondamentaux de l'intelligence artificielle, du machine learning et du deep learning, en nous concentrant particulièrement sur les réseaux de neurones convolutifs. Nous aborderons également les bases de la vision par ordinateur et la méthodologie YOLO, avec un accent spécifique sur YOLOv8, utilisée pour la détection et la classification des objets. Enfin, nous examinerons comment organiser les datasets, ce qui est crucial pour la précision et l'efficacité des modèles de détection et de classification.

II. Intelligence Artificielle :

II.1 Définition :

L'intelligence pourrait être considérée comme un ensemble de techniques permettant aux machines d'exécuter des tâches et de résoudre des problèmes normalement réservés à l'homme et à certains animaux.

Les tâches d'intelligence artificielle sont parfois très simples pour l'homme, telles que reconnaître et localiser des objets dans une image, planifier les mouvements d'un

robot pour rattraper un objet ou conduire une voiture. Ils nécessitent parfois une planification complexe, telle que jouer aux échecs. Les tâches les plus complexes nécessitent beaucoup de connaissances et de bon sens, par exemple pour traduire un texte ou mener un dialogue. Ces dernières années, l'intelligence a presque toujours été associée à des capacités d'apprentissage. C'est grâce à l'apprentissage qu'un système intelligent capable d'effectuer une tâche peut améliorer ses performances avec l'expérience. C'est par l'apprentissage qu'il peut apprendre à effectuer de nouvelles tâches et à acquérir de nouvelles compétences. Le domaine de l'IA n'a pas toujours considéré l'apprentissage comme essentiel à l'intelligence. Construire un système intelligent consistait autrefois à écrire un programme "à la main" pour jouer aux échecs, pour reconnaître les caractères imprimés ou pour poser un diagnostic médical à partir des symptômes. Mais cette approche « manuelle » a ses limites.

II.2 Relation entre l'intelligence artificielle, le ML et le Deep Learning :

Le Deep Learning est un nouveau domaine de recherche de la ML, qui a été introduit dans le but de rapprocher le ML de son objectif principal à savoir : l'intelligence artificielle. Il concerne les algorithmes inspirés par la structure et du fonctionnement du cerveau. Ils peuvent apprendre plusieurs niveaux de représentation dans le but de modéliser des relations complexes entre les données.

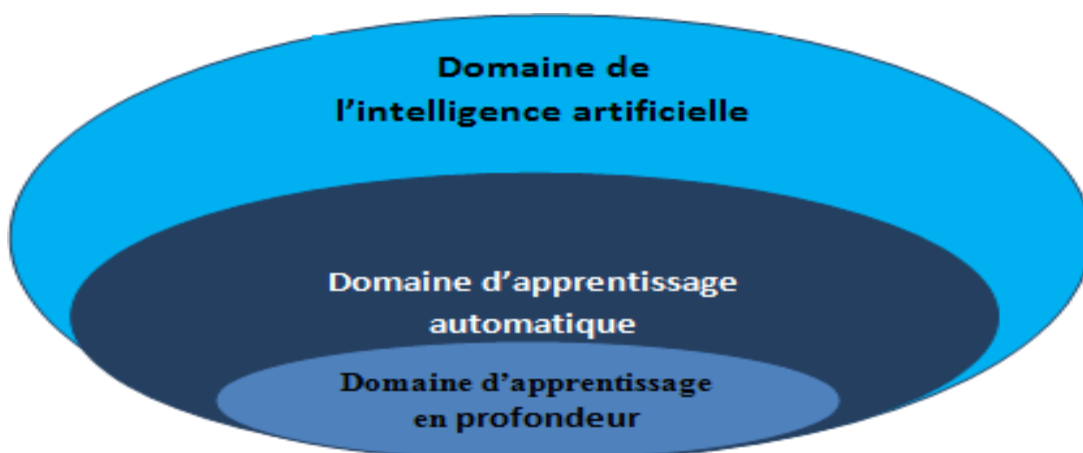


Figure 11 : La relation entre l'IA, le ML et le Deep Learning.

III. Machine Learning (ML) :

Le ML désigne la science de la création et de l'étude d'algorithmes qui améliorent leur propre comportement de manière itérative par la conception.

À l'origine, le domaine était consacré au développement de l'intelligence artificielle, mais en raison des limites de la théorie et de la technologie qui existaient, il est devenu plus logique de concentrer ces algorithmes sur des tâches spécifiques. La plupart des algorithmes (ML) tels qu'ils existent aujourd'hui se concentrent sur l'optimisation des fonctions.

Ainsi, l'utilisation d'algorithmes (ML) devient souvent un processus répétitif d'essais et d'erreurs, dans lequel le choix de l'algorithme parmi les problèmes donne des résultats de performance différents. C'est très bien dans certains contextes, mais dans le cas de la modélisation du langage et de la vision par ordinateur, cela devient problématique.

Après les progrès des capacités théoriques et technologiques dont nous disposons aujourd'hui, l'apprentissage approfondi (Deep Learning) est apparu et se développe rapidement comme l'un des domaines scientifiques les plus passionnants. Il est utilisé dans des technologies telles que les voitures à conduite autonome, la reconnaissance d'images sur les plateformes de médias sociaux et la traduction de textes d'une langue à l'autre.

IV. Deep Learning (DL) :

IV.1 Définition :

Le Deep Learning est une sous-branche de l'apprentissage automatique qui utilise des réseaux neuronaux artificiels pour modéliser des données complexes. Inspiré par le fonctionnement du cerveau humain, le Deep Learning utilise des couches de neurones artificiels pour extraire des caractéristiques et des modèles à partir de données brutes.

IV.2 Les réseaux de neurones convolutifs :

Les réseaux de neurones convolutifs sont directement inspirés du cortex visuel des vertébrés. Un réseau de neurones à convolution, appelé aussi convnet, ou encore CNN se représente en général sous la forme suivante :

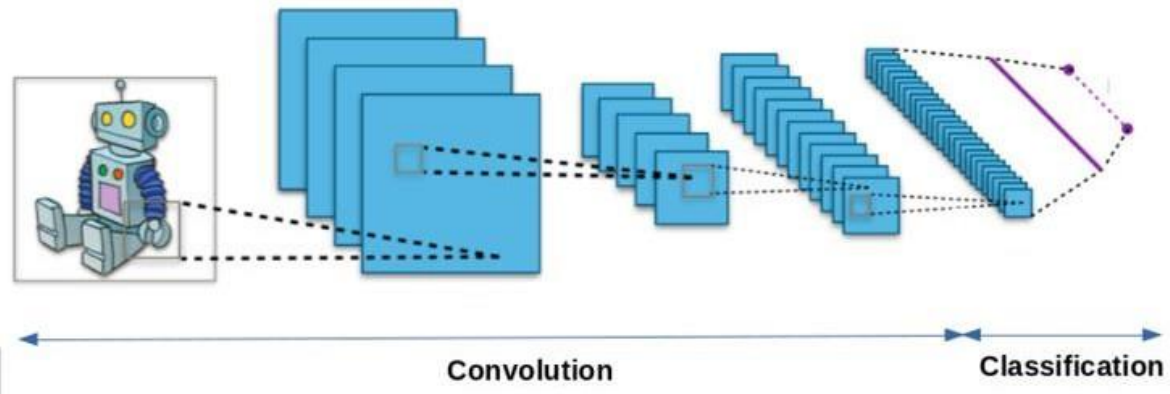


Figure 12 : Réseau de neurones convolutifs.

IV.2.1 Architecture d'un réseau de neurones convolutifs :

On distingue deux parties, une première partie que l'on appelle la partie convolutive du modèle et la seconde partie, que l'on va appeler la partie classification du modèle qui correspond à un modèle MLP.

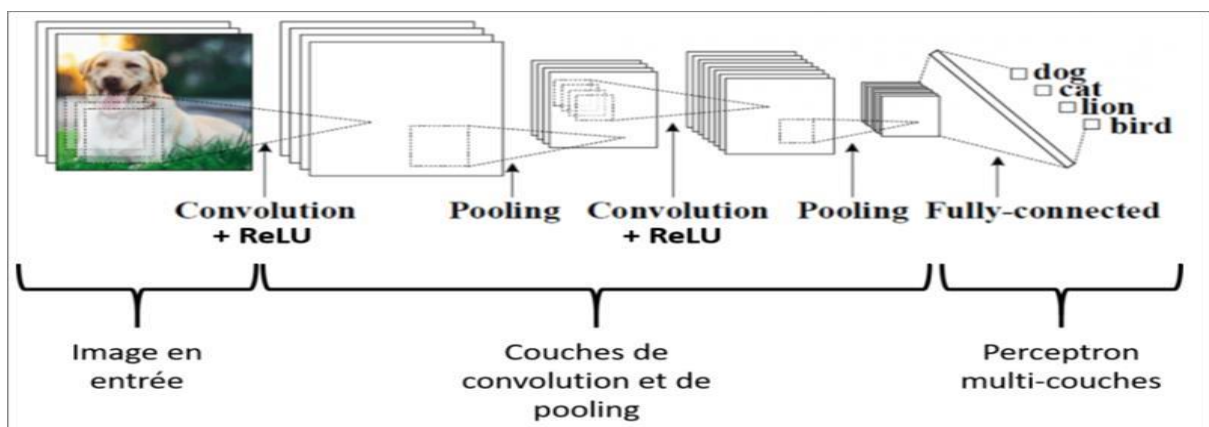


Figure 13 : Architecture d'un CNN.

IV.2.2 Caractéristiques et avantages des CNN :

Un avantage majeur des CNN est l'utilisation d'un poids unique associé aux signaux entrants dans tous les neurones d'un même noyau de convolution. Cette méthode réduit l'empreinte mémoire, améliore les performances et permet une invariance du

traitement par translation. C'est le principal avantage du CNN par rapport au **perceptron multicouche**, qui, lui, considère chaque neurone indépendant et affecte donc un poids différent à chaque signal entrant.

Lorsque le volume d'entrée varie dans le temps, il devient intéressant de rajouter un paramètre le long de l'échelle de temps dans le paramétrage des neurones. On parlera dans ce cas de réseau neuronal à retard temporel.

Comparés à d'autres algorithmes de classification d'images, les CNN utilisent relativement peu de prétraitement. Cela signifie que le réseau est responsable de faire évoluer tout seul ses propres filtres, ce qui n'est pas le cas d'autres algorithmes plus traditionnels. L'absence de paramétrage initial et d'intervention humaine est un atout majeur des CNN.

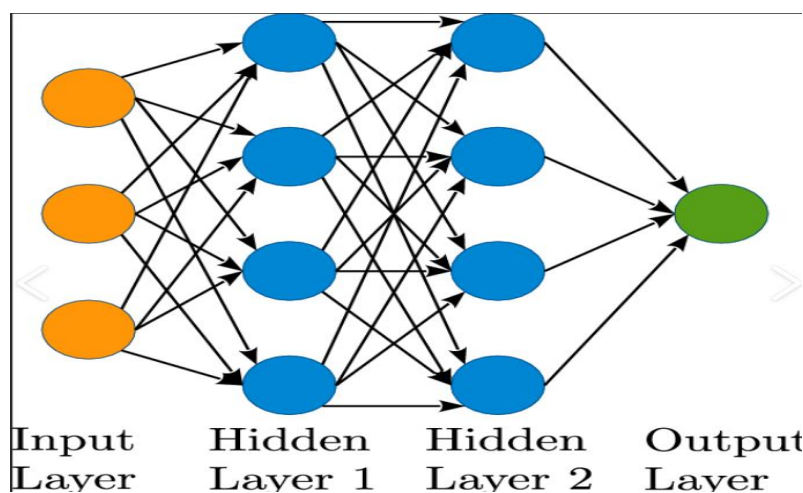


Figure 14 : Split

IV.2.2.1 Modèle génératif :

Si les modèles discriminatifs comme (CNN, RNN) sont utilisés pour prédire les données du label et de l'entrée, le modèle génératif décrit comment générer les données, il apprend et fait des prédictions en utilisant la loi de Bayes. Cependant les modèles génératifs sont capables de bien plus que la simple classification comme par exemple générer de nouvelles observations. Voici quelques exemples de modèle génératif :

- Boltzmann Machines.
- Restricted Boltzmann Machines.
- Deep Belief Networks.
- Deep Boltzmann Machines.
- Generative Adversarial Networks.
- Generative Stochastic Networks.

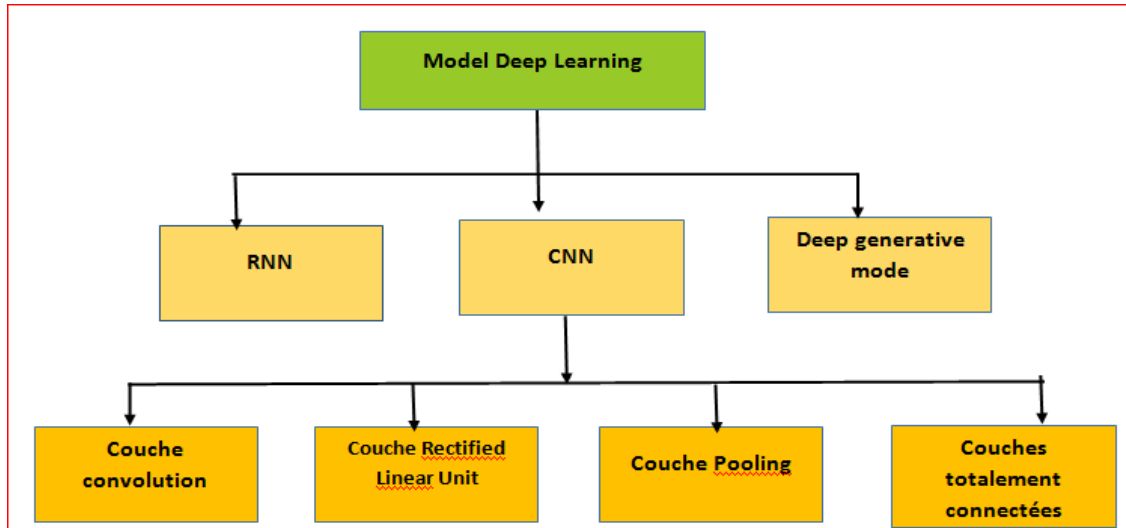


Figure 15 : résume les différents modèles du deep Learning.

IV.2.2.2 Convolutions :

I.1.1.1.1. Convolution d'image :

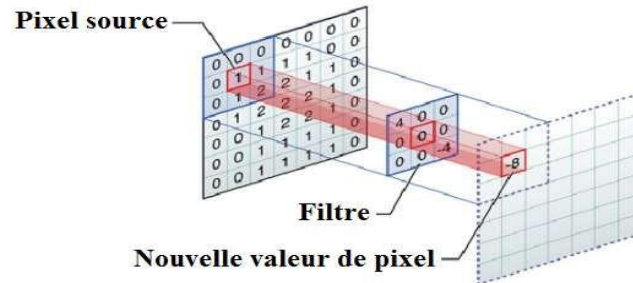
- Opération linéaire entre une image et un filtre (kernel), produisant une nouvelle image.
- Soit une image I , un filtre K de taille $F \times F$, l'image calculée est le recouvrement de l'image avec le filtre de toutes les façons possibles :

$$\hat{I}(i, j) = I(i, j) * K(i, j) = \sum_{n=0}^{F-1} \sum_{m=0}^{F-1} K(n, m) \cdot I(i + n, j + m) \quad (1)$$

La sortie $\hat{I}$ est appelée **carte** de caractéristiques.

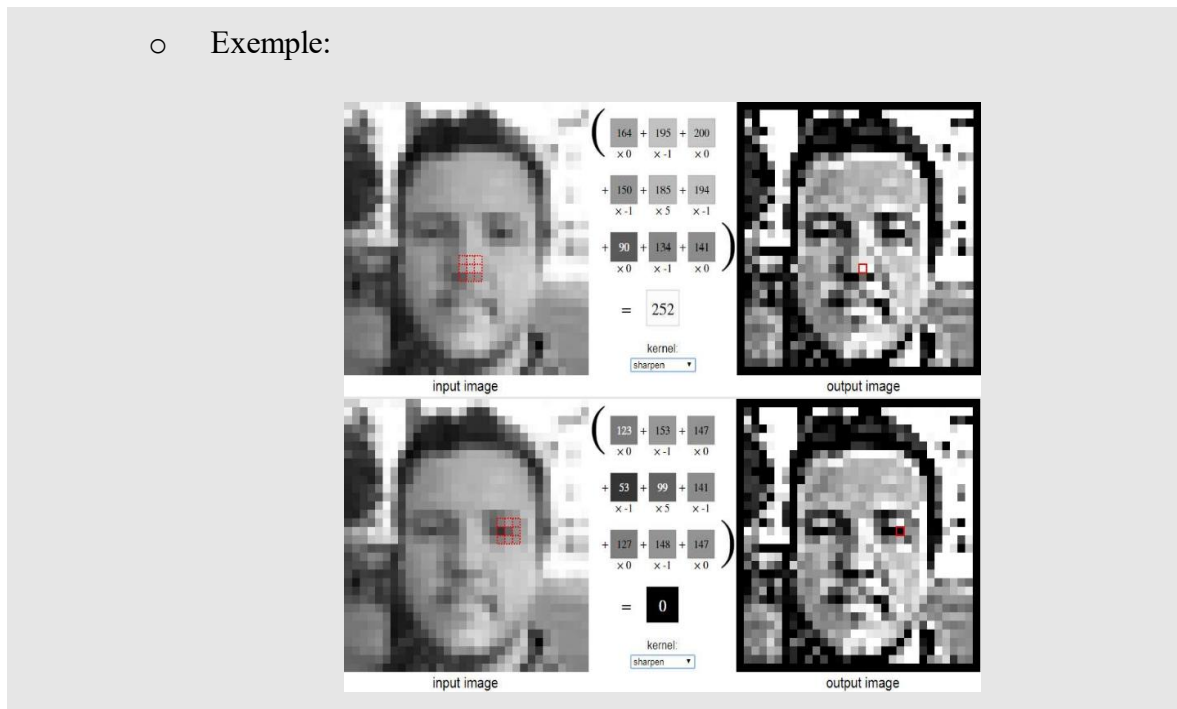
Chaque pixel de la carte se calcule comme une somme des poids du filtre multipliés un à un aux pixels correspondants dans le voisinage de ce pixel.

- Exemple:



- Filtres (ou noyaux de convolution), expliqués visuellement par Victor Powell setosa.io/ev/image-kernels

- Exemple:



IV.2.2.3 Paramètres du filtre :

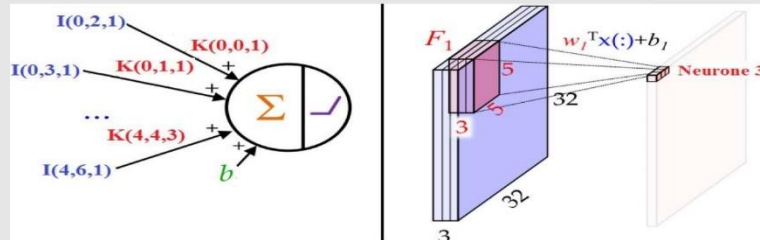
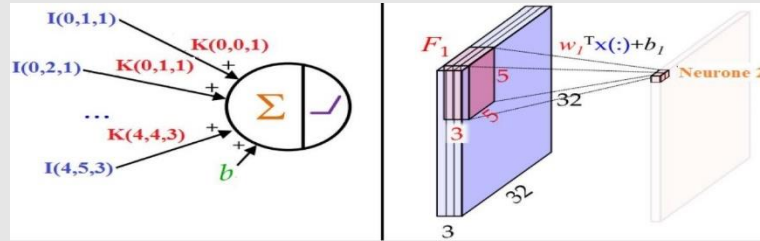
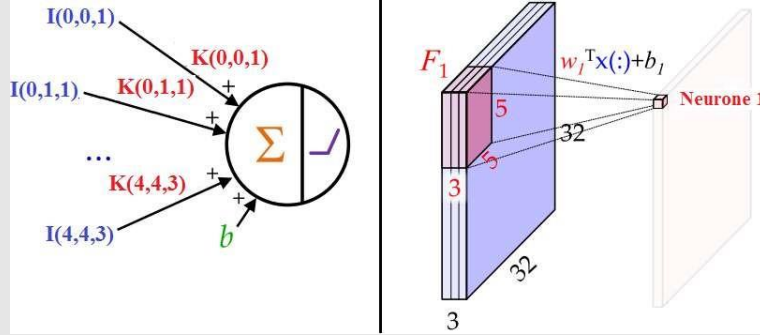
IV.2.2.3.1 Dimensions d'un filtre :

Un filtre de taille $F \times F$ appliqué à une image contenant C canaux est un volume de taille $F \times F \times C$ qui effectue des convolutions sur une

image de taille $N \times M \times C$ et qui produit une carte de caractéristiques de taille $H \times L \times 1$.

$$\hat{I}(i, j) = I(i, j) * K(i, j) = \sum_{k=1}^C \sum_{n=0}^{F-1} \sum_{m=0}^{F-1} K(i, j, k) \cdot I(i + n, j + m, k) \quad (2)$$

○ Point de vue d'un neurone :

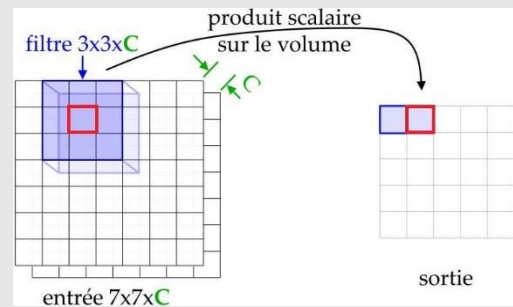
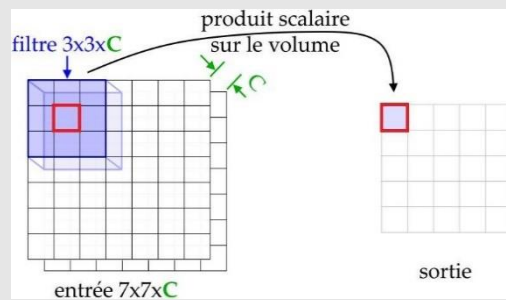


IV.2.2.3.2 Stride :

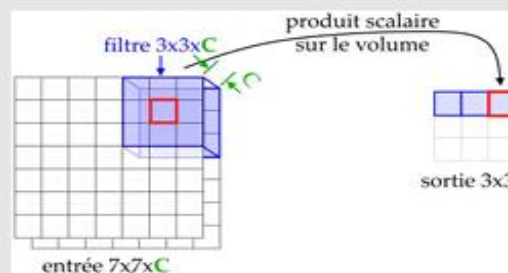
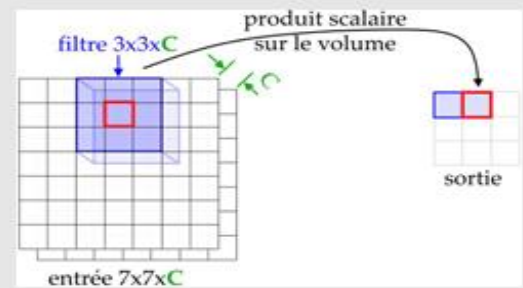
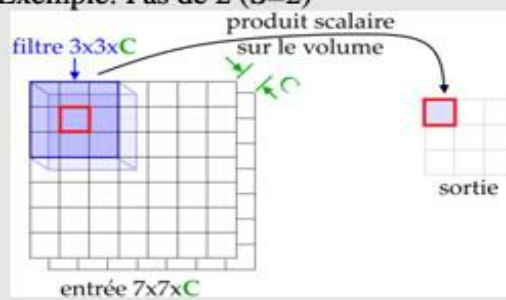
- La stride S (le pas de convolution) est un paramètre qui dénote le nombre de pixels par lesquels la fenêtre se déplace après chaque opération.
- S généralement 1 ou 2, rarement plus grand.
- S Réduit la taille de sortie $L = \frac{M-F}{S} + 1$ et $H = \frac{N-F}{S} + 1$

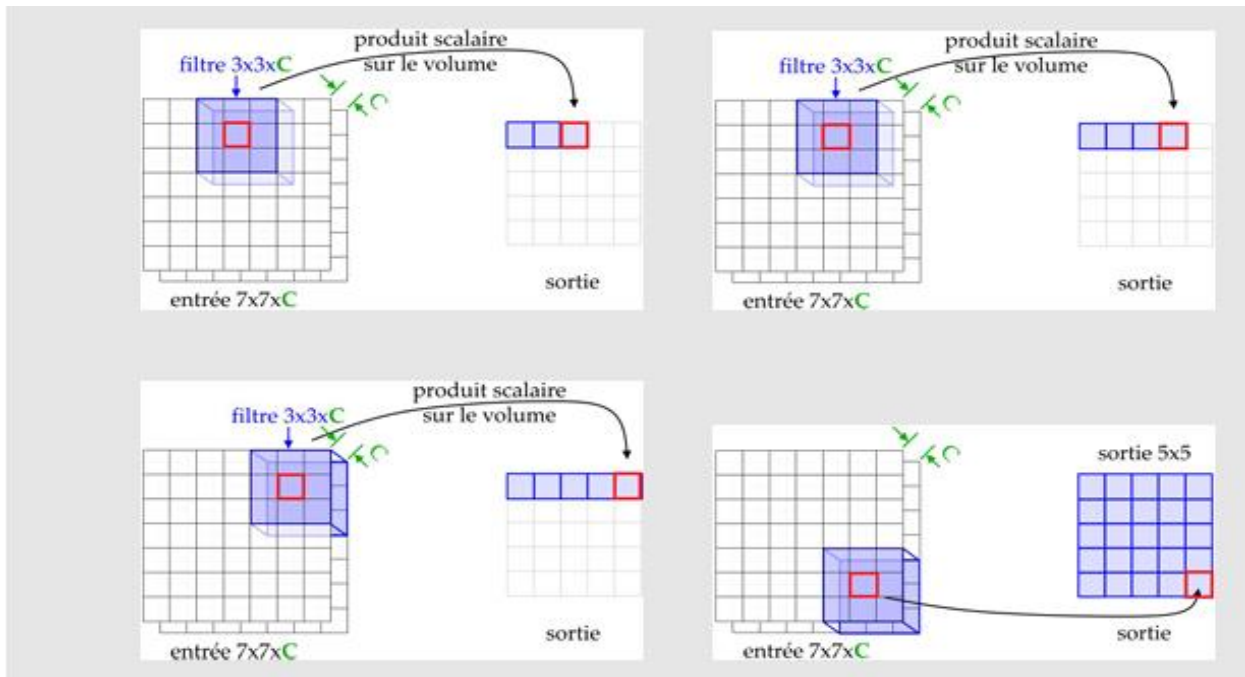
➤ Problème : convolution impossible pour les pixels en bordure.

○ Exemple : Pas de 1 (S=1):



□ Exemple: Pas de 2 (S=2)



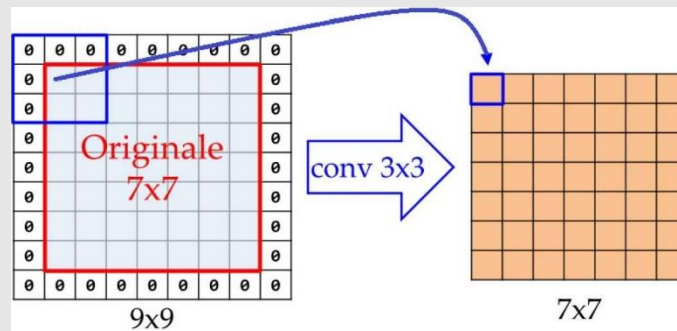


IV.2.2.3.3 Remplissage (Padding) :

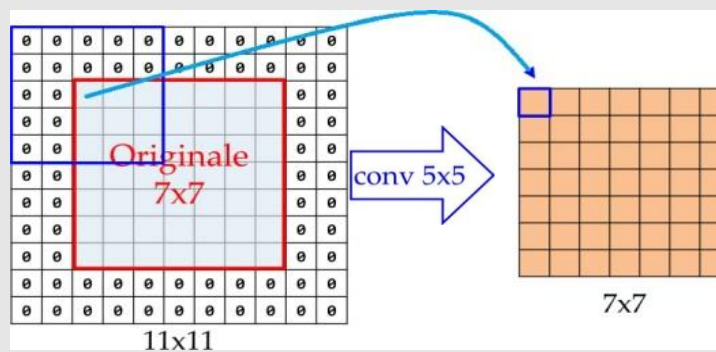
- Solution : Agrandir l'image initiale :
 - Plusieurs approches possibles :
 - Remplissage par zéros (Zero-padding): Consiste à ajouter P zéros à chaque côté des frontières l'entrée.
 - * Pour un filtre de taille $F \times F$, padding de largeur $P = \frac{F-1}{2}$
 - * Pour filtre 5×5 , padding de largeur 2
 - * Pour filtre 7×7 , padding de largeur 3
 - Duplication : Consistant à dupliquer les premiers et les dernières lignes et colonnes au-delà des bords.
 - * Le nombre de ligne/colonne à ajouter dépend de la taille du filtre.
 - * La taille d'activation une carte de caractéristiques dépend du padding.

$$L = \frac{M-F+2P}{S} + 1 \text{ et } H = \frac{N-F+2P}{S} + 1 \quad (3)$$

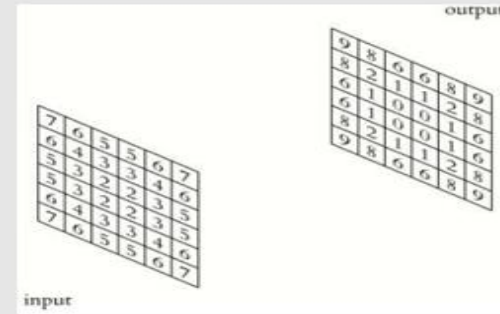
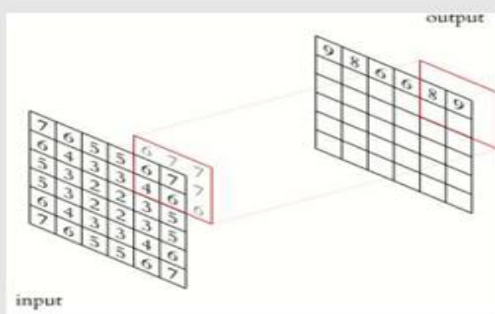
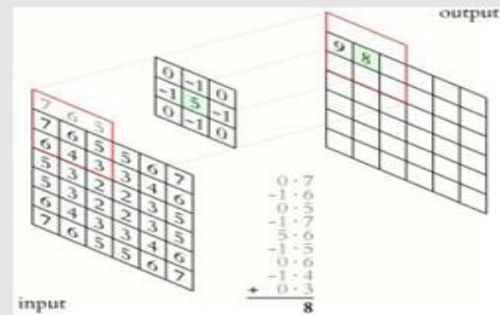
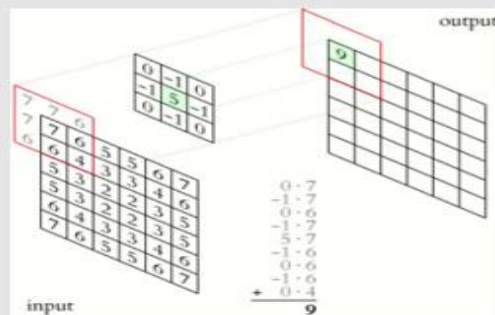
- Exemple: Filtre 3x3, padding de $P = 1$:



- Exemple: Filtre 5x5, padding de $P = 2$:



□ Exemple: Duplication

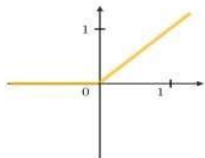
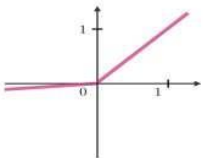
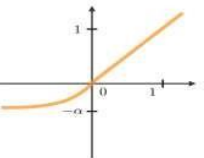


IV.2.2.3.4 Fonctions d'activation :

Une correction peut être effectuée entre les couches de convolution et de pooling
Application d'une fonction d'activation ReLU (Unité linéaire rectifiée).

ReLU est une fonction d'activation y qui est utilisée sur tous les éléments du volume. Elle a pour but d'introduire des complexités non-linéaires au réseau. Ses variantes sont récapitulées dans le tableau suivant :

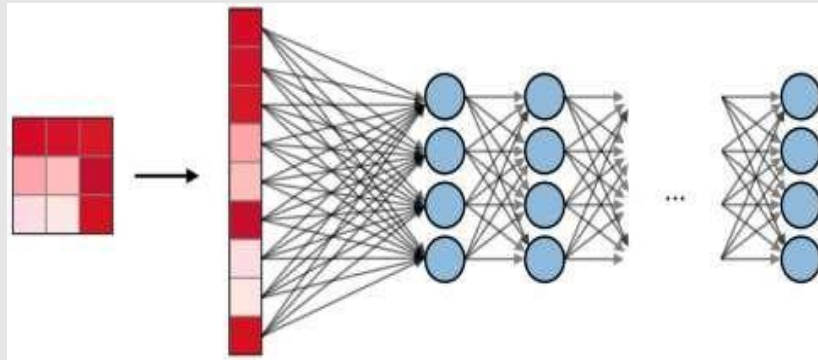
○ Exemple :

ReLU	Leaky ReLU	ELU
$g(z) = \max(0, z)$	$g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$	$g(z) = \max(\alpha(e^z - 1), z)$ with $\alpha \ll 1$
		
Complexités non-linéaires interprétables d'un point de vue biologique.	Répond au problème de <i>dying ReLU</i>	Dérivable partout

IV.2.2.3.5 Couche totalement connectée :

- La Couche totalement connectée (TC) s'applique lorsque chaque entrée est connectée à tous les neurones.
- Les couches de TC sont typiquement présentes à la fin des architectures de CNN et peuvent être utilisées pour optimiser des objectifs tels que les scores de classe.
- Un cas particulier important : lorsque la taille de filtre coïncide avec la taille d'entrée, on a affaire à une couche TC.

O Exemple:



IV.3 Révolution d'apprentissage profond :

IV.3.1 L'évolution des CPU et des GPU :

L'amélioration des processeurs, notamment les GPU, a été un catalyseur majeur dans l'avancée des réseaux de neurones en fournissant la puissance de calcul nécessaire pour entraîner des modèles profonds et complexes.

IV.3.2 Une Activation du GPU :

Nous devons activer l'accélération GPU dans notre Google Colab afin que le système YOLO.v3 puisse traiter les détections plus de 100 fois plus vite que le CPU.

- Le GPU utiliser par Cloud : TESLA P4 Pour cela il y a des étapes à faire :
- Cliquons sur Modifier (Edit) en haut à gauche de notre carnet (Notebook).
- Cliquons sur Paramètres (Notebook settings) dans la liste.
- Sélectionnons GPU et appuyez sur "Save".

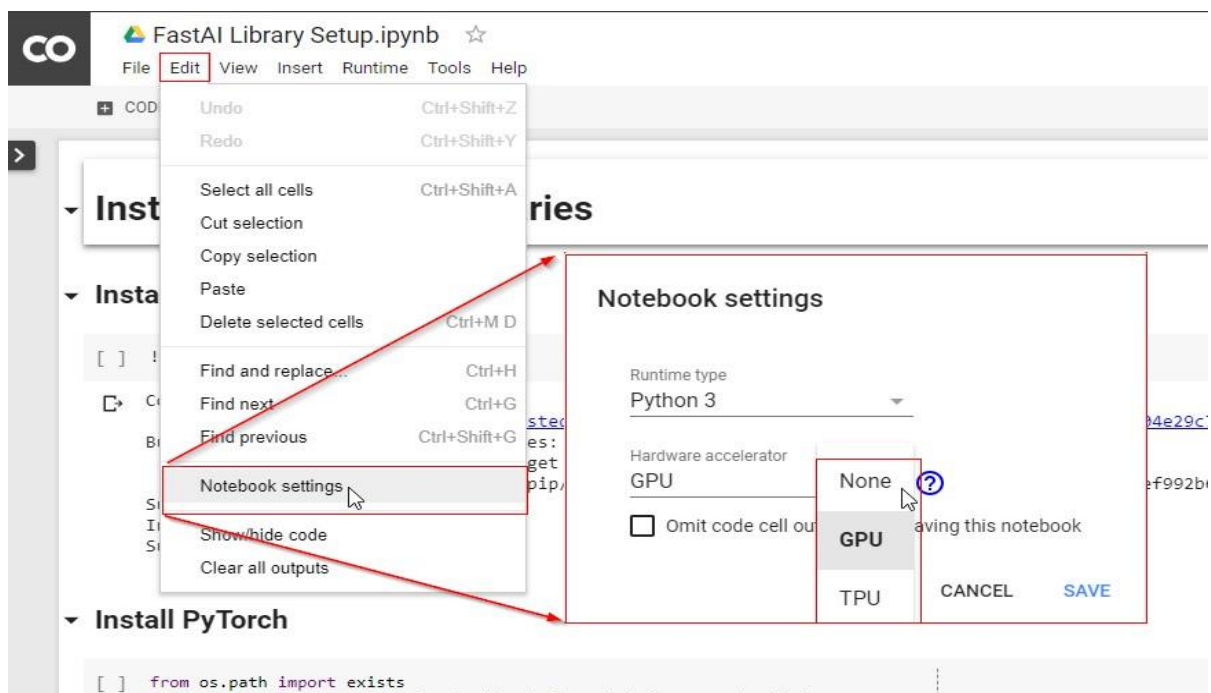


Figure 16 : résume les différents modèles du deep Learning.

v. Vision par ordinateur :

La vision par ordinateur est un ensemble de techniques conçues pour l'interprétation automatique d'images. Au début de l'année 1964, les experts en intelligence artificielle se sont concentrés sur la possibilité de simuler les capacités sensorielles humaines.

Comme la vision est le sens humain le plus important, il est naturel que les travaux dans cette direction aient été nombreux. La démocratisation des appareils photo numériques a encore accéléré les possibilités offertes par le traitement des images, non seulement grâce.

Aux processus de correction intégrés dans les capteurs eux-mêmes, mais également grâce à la nouvelle facilité de post-traitement informatique.

La vision par ordinateur est l'émulation de la capacité du cerveau humain à interpréter une scène dynamique à partir de signaux visuels, en particulier pour identifier rapidement des objets et anticiper leurs mouvements. Ces fonctions cognitives sont essentielles pour la navigation en robotique autonome, mais profitent également aux applications d'exploration de données.

La numérisation automatique de vieux documents, la recherche d'images similaires en ligne ou la description audio automatique sont des exemples de tâches pouvant s'appuyer sur une brique d'interprétation des images. En particulier, les efforts de la communauté de la vision par ordinateur se sont concentrés sur la reconnaissance des objets dans les images, y compris leur identification et leur localisation.

De nombreux descripteurs ad hoc ont été introduits pour des applications aussi diverses que la détection de visage, la classification automatique de photographies d'animaux ou la reconnaissance optique de caractères.

Le dénominateur commun de ces œuvres est d'essayer de donner un sens aux images. L'extraction de la sémantique à partir d'informations enregistrer sur une base de données constitue également l'objectif de la reconnaissance d'une scène.

Ce projet se situe à l'intersection de la vision par ordinateur et de l'apprentissage automatique. En particulier, nous proposons de mettre en œuvre des méthodes d'apprentissage en profondeur pour l'interprétation automatique des images de reconnaissance d'une scène.

Dans la [figure 17] nous montrons les domaines les plus connus de la vision par ordinateur ainsi que la différence entre eux et leurs multiples niveaux.



Figure 17 : Les domaines de la vision par ordinateur.

Dans la [figure 18] nous donnons un exemple d'une approche de vision par ordinateur dans le domaine médical suivi d'un schéma qui montre les différentes étapes pour réaliser cette approche

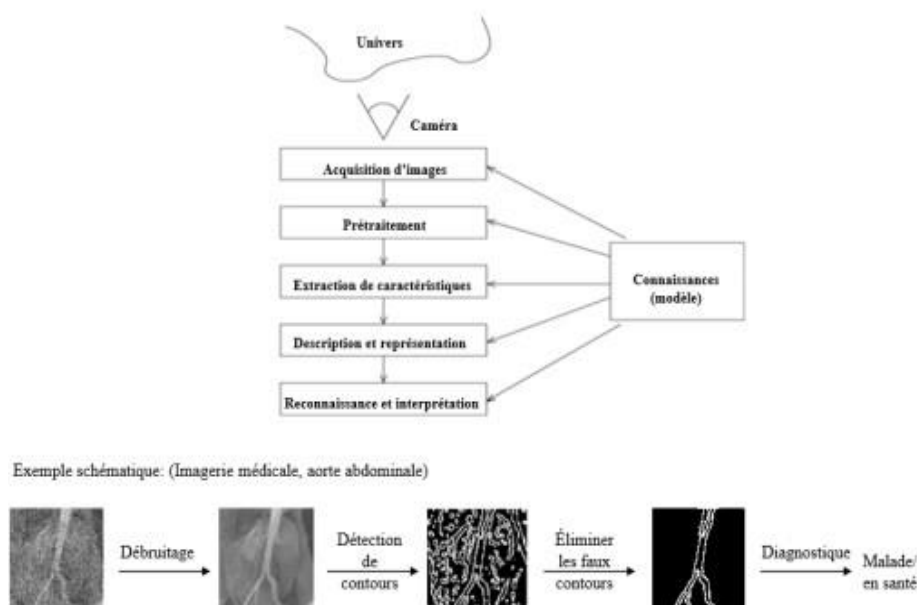


Figure 18 : Schéma représentant l'approche de vision par ordinateur.

VI. K- means :

VI.1 Définition et principes :

C'est l'un des algorithmes le plus connu et le plus utilisé en classification non supervisée du fait de sa simplicité de mise en œuvre. Il vise à produire un partitionnement des pixels de manière à ce que les pixels d'une même classe soient semblables et les pixels issus de deux classes différentes soient dissemblables. Chaque classe de la partition est définie par ses objets et son centroïde.

Le principe de K-means est de choisir aléatoirement un ensemble de centres fixé a priori et de chercher itérativement la partition optimale. Chaque individu (également appelé centroïde ou centroid en anglais) est associé au centre le plus proche, après

l'affectation de toutes les données la moyenne de chaque classe est calculée, elle constitue les nouveaux représentants des groupes, lorsqu'on abouti à un état stationnaire (aucune donnée ne change de groupe) l'algorithme est arrêté.

Le principe des K-means est de minimiser la fonction (4) objective :

$$J(x, v) = \sum_{i=1}^N \sum_{k=1}^c \|x_i - v_k\|^2 \quad (4)$$

- c est le nombre de classe.
- N est le nombre de données.
- n est le nombre de donnée de chaque classe.
- v_k est le centre de classe i .
- $\|\cdot\|$: est la norme Euclidienne qui mesure la similarité entre un pixel est un autre.

Tenant compte de la nouvelle fonction objectif (4), la mise à jour des centres des classes v se fait avec l'équation (5) :

$$v_k = \frac{1}{n} \sum_{i=1}^n x_i \quad (5)$$

L'algorithme suivant résume les étapes principales pour l'exécution :

Etape 1 : Choisir aléatoirement les C centres initiaux $v_1, v_2, v_3, \dots, v_C$ à partir des point $\{x_1, x_2, \dots, x_N\}$.

Etape 2 : Assigner $x_i \ i = 1 \dots N$ à la classe $k; k = 1 \dots c$ si :
 $\|x_i - v_k\| \leq \|x_i - v_p\| ; p = 1, \dots, C$ Avec :
 $i \neq p$.

Etape 3 : Mettre à jour les centres des classes par la formule (5).

Etape 4 : Répéter de l'étape 2 et 3 jusqu'à atteindre la convergence $v_k^* = v_k ; k = 1, \dots, c$.

VI.2 Avantages de l'algorithme :

- L'algorithme de k-means est très populaire du fait qu'il est très facile à comprendre et à mettre en œuvre.
- Sa simplicité conceptuelle et sa rapidité
- Applicable à des données de grandes tailles, et aussi à tout type de données (même textuelles), en choisissant une bonne notion de distance.

VI.3 Cet algorithme à plusieurs inconvénients :

- La classification finale dépend du choix de la partition initiale et le choix aléatoire des centres initiaux.
- Convergence vers un minimum local.
- L'algorithme est fortement sensible au bruit.

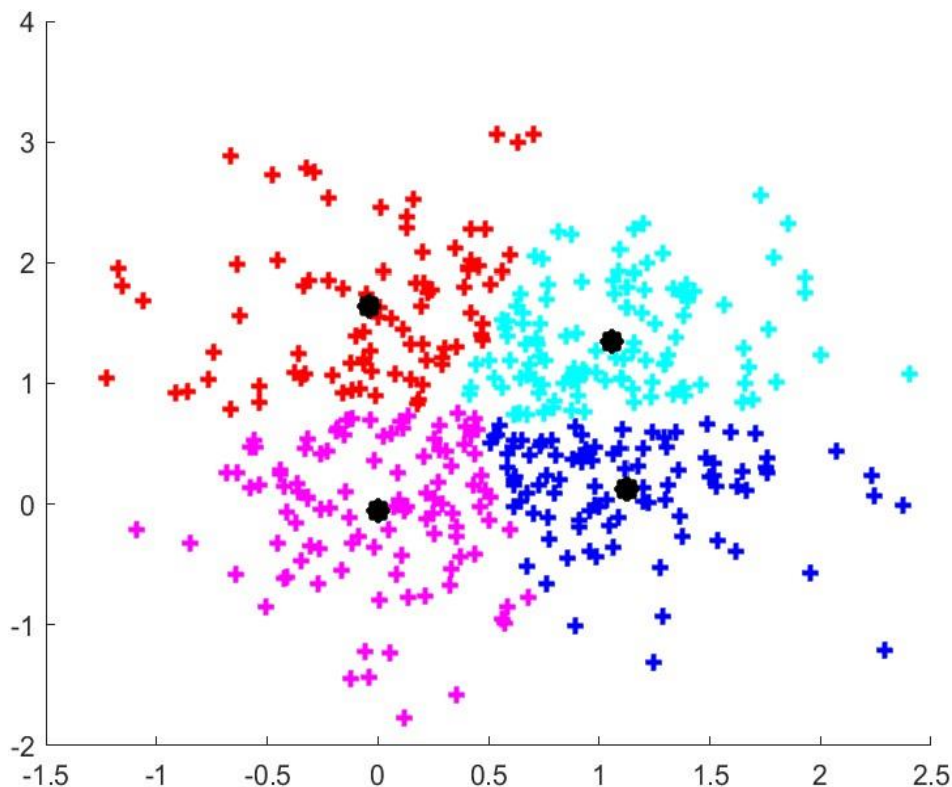


Figure 19 : la représentation de k-means avec $k=4$.

VII. Environnement de développement :

VII.1 Environnement matériel (hardware) :

Un ordinateur avec les caractéristiques suivantes :

- **Processeur** : Intel i5 8550u Ram
- **RAM** : 8go
- **Disque Dur** : 1000 go HDD Webcam intégrée de marque

VII.2 Environnement immatériel (software) :

- **Système d'exploitation** : Windows 11 64bit
- **Logiciel** : python 3.7

VIII. YOLO (You Only Look Once) :

VIII.1 Définition (Yolo, c'est quoi?) :

YOLO, ou "You Only Look Once", est un outil spécial qui aide les ordinateurs à voir rapidement et avec précision les choses dans les images, les fichiers textes ou les vidéos.

Créé par les experts Joseph Redmon et Ali Farhadi en 2015, YOLO est plus rapide que les outils plus anciens car il analyse l'image entière en une seule fois. Cette vérification rapide permet à YOLO d'identifier rapidement s'il y a d'autres objets, comme des voitures, des arbres ou des animaux, et où ils se trouvent dans l'image.

VIII.2 Les versions de YOLO :

Les créateurs de YOLO continuent de le mettre à jour l'algorithme pour l'améliorer continuellement ; il existe de nombreuses versions, de YOLOv1 à YOLOv9 (le plus récent, sorti en Février 2024), chaque nouvelle version étant plus rapide et précise. YOLO est devenu très populaire car il donne aux machines des superpouvoirs pour

voir et comprendre le monde rapidement et repérer des objets pour une multitude d'applications dans le monde réel.

VIII.2.1 YOLO V1 :

La première version de YOLO a révolutionné la communauté des chercheurs en IA / Computer Vision avec ses capacités de détection d'objets en temps réel, offrant des vitesses d'inférence bien plus rapides que les méthodes existantes telles que R-CNN.

YOLO v1 divise l'image entrante en une grille et prédit plusieurs boîtes englobantes et probabilités de classe pour chaque cellule de la grille.

- Cependant, avec cette première version, la précision était un compromis. YOLO avait alors du mal avec les petits objets et produisait de nombreuses erreurs de localisation d'objet.

VIII.2.2 YOLO V2 et V3:

Les versions suivantes, telles que YOLO v2 et v3, ont introduit des améliorations notables et de nouvelles fonctionnalités comme les boîtes d'ancrage, utilisant le clustering k-means pour prédire des coordonnées de boîtes englobantes plus précises.

Ces versions ont également profité de la normalisation par lots et la gestion d'images d'entrée de plus haute résolution, conduisant à des performances de détection nettement meilleures sur des benchmarks tels que les ensembles de données Pascal VOC et COCO.

VIII.2.3 YOLO V4 et V5:

Avec pour objectif d'atteindre à la fois une grande vitesse et une haute précision, YOLO v4 a introduit des fonctionnalités telles que le pooling pyramidal spatial et une architecture YOLO plus complexe basée sur des réseaux convolutionnels de pointe.

YOLO v5 s'est quant à lui concentré sur la simplification et l'optimisation, lui permettant de fonctionner extrêmement rapidement sur du matériel moins puissant tout en maintenant une haute précision.

VIII.2.4 YOLO V6 à V8:

Les versions les plus récentes de YOLO, à partir de la version 6, introduisent des améliorations continues axées sur les applications concrètes de YOLO, telles que les véhicules autonomes ou la surveillance vidéo. Plus on avance dans le temps, et plus YOLO s'éloigne de la communauté des chercheurs pour atteindre le grand public et les cas d'usage appliqués à la vie réelle.

Ces versions ont affiné l'utilisation des techniques de Deep Learning, y compris diverses formes d'augmentation des données et d'algorithmes d'optimisation qui ont aidé à améliorer la précision moyenne et la capacité à détecter une gamme diversifiée de classes d'objets.

VIII.2.5 YOLO V9:

Le 21 février 2024, Chien-Yao Wang, I-Hau Yeh et Hong-Yuan Mark Liao ont publié l'article "YOLOv9 : Apprendre ce que vous voulez apprendre en utilisant l'Information de Gradient Programmable", qui introduit une nouvelle architecture de modèle de vision par ordinateur YOLOv9.

YOLOv9 représente une avancée majeure dans la série des modèles YOLO, offrant des améliorations significatives en termes de précision et d'efficacité pour la détection d'objets en temps réel. Il se distingue de ses prédécesseurs, notamment YOLOv8, par une réduction de 49% du nombre de paramètres et de 43% de la complexité computationnelle, tout en augmentant la précision moyenne sur le jeu de données MS COCO de 0,6%.

Tableau 1 : Des différentes versions et évolutions de YOLO.

Version	Amélioration	Compromis Vitesse / Précision	Application
V1	Précision par cellules de grille, méthode à tir unique	Rapide mais moins précis	Détection en temps réel fondamentale (recherche)
V2 et V3	Boîtes d’ancrage, normalisation par lots	Plus rapide et plus précis	Diverses applications en temps réel
V4 et V5	Pooling pyramidal spatial, optimisations	Equilibre entre vitesse et précision	Environnements exigeants, comme le transport
V6 à V8	Optimisations ciblées, architectures améliorées	Très précis et en temps réel	Applications spécialisées, comme la surveillance
V9	Amélioration de la détection des petits objets, intégration avec d’autres modèles d’IA et IA explicable	Précision et vitesse accrues	Applications telles que l’imagerie médicale, la conduite autonome ou la détection de défauts industriels

VIII.3 Fonctionnement de YOLO:

Voici comment l'algorithme de détection d'objets YOLO (You Only Look Once) fonctionne, expliqué en étapes simples :

VIII.3.1 Prendre une photo :

Tout d'abord, l'algorithme YOLO commence par une image, tout comme lorsque vous prenez une photo avec un appareil photo. C'est ce que nous appelons la détection d'objets basée sur la classification d'images !

VIII.3.2 Diviser l'image :

Ensuite, il divise l'image donnée en petits carrés, comme un damier. Chaque carré sera vérifié pour voir s'il contient un objet (un chat, un chien ou encore une boîte de conserve, par exemple).

VIII.3.3 Chercher des indices :

Pour chaque carré, YOLO recherche des indices ou des caractéristiques comme des bords, des formes ou des textures qui pourraient indiquer quel objet s'y trouve. Il les entoure avec des boîtes englobantes. Comme YOLO a besoin d'apprendre pour bien comprendre un nouveau jeu de données et interpréter, on lui a parfois donné un jeu de données de référence (ou "vérité terrain") dans lequel il peut puiser pour avoir des points de comparaison.

VIII.3.4 Faire des prédictions :

L'algorithme fait une supposition pour chaque carré d'une image : quel objet cela pourrait-il être et où exactement se trouve-t-il dans le carré ? Il attribue à chaque supposition un score pour montrer son niveau de certitude.

VIII.3.5 Éliminer les excédents :

Certains carrés ont des suppositions de différents objets qui se chevauchent, comme deux carrés devinant une partie de la même voiture. YOLO choisit la meilleure supposition pour chaque objet, en se débarrassant des suppositions superflues.

VIII.3.6 Montrer ce qu'il a trouvé :

À la fin, YOLO vous montre où il pense que chaque objet se trouve en dessinant des boîtes autour d'eux et en les étiquetant, comme "voiture" ou "arbre". Si vous lui donnez 1'000 images contenant des chiens et des chats, et lui dites d'identifier les chats, il vous montrera des images enrichies de métadonnées pointant vers les chats.

VIII.4 Evolution de la détection d'objets : de YOLO 1 à YOLO 9 :

YOLO, acronyme de "You Only Look Once", est un algorithme de détection d'objets en temps réel qui a connu d'importantes améliorations depuis sa création. En tant que détecteur "one shot", il traite les images et identifie les objets en prédisant des boîtes englobantes et des probabilités de classe en une seule passe. Au fil du temps, YOLO est devenu de plus en plus résilient et performant, comme l'illustre très bien la dernière publication de ses auteurs :

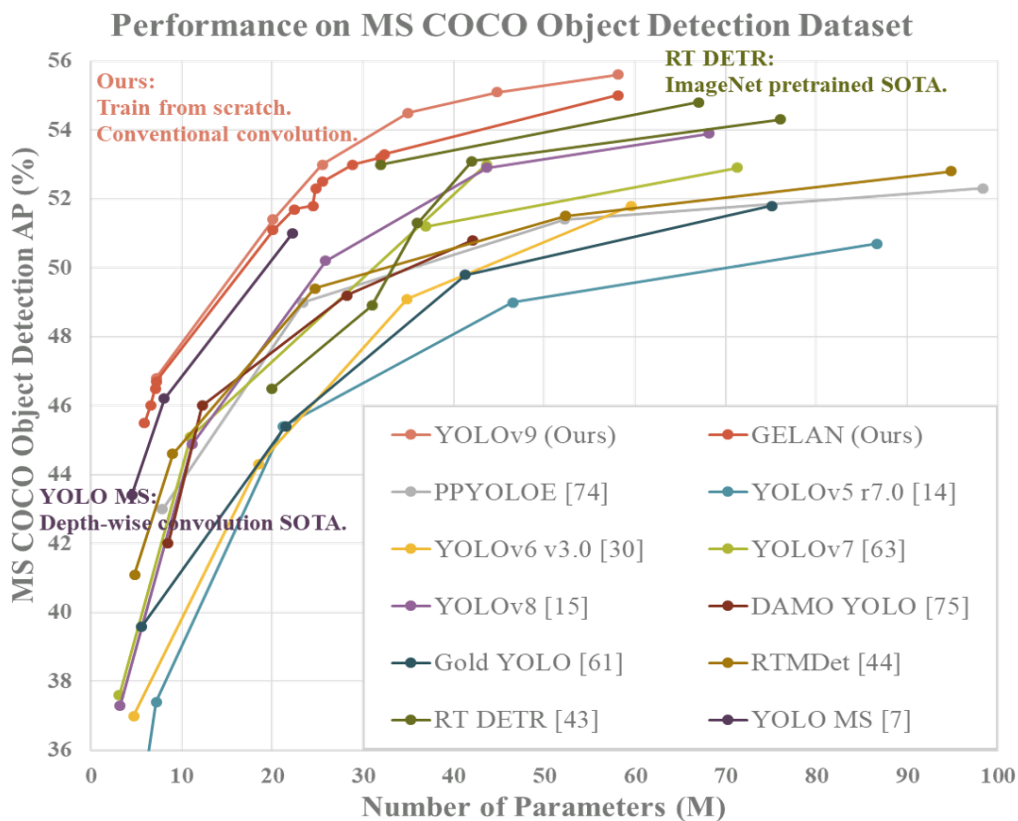


Figure 20 : Illustration de la performance de YOLO .

VIII.5 La version utilisée – YOLOv8 :

Dans notre projet, nous avons utilisé YOLOv8. C'est la dernière version du célèbre modèle de détection d'objets et de classification d'images en temps réel. S'appuie sur des avancées de pointe en matière d'apprentissage profond et de vision par ordinateur,

offrant des performances inégalées en termes de rapidité et de précision. YOLOv8 peut réaliser la détection et la localisation d'objets en une seule étape.

VIII.5.1 YOLOv8 pour la Détection d'Objets :

Identifier et localiser plusieurs objets dans une image. Pour chaque objet détecté, le modèle prédit une boîte englobante et une classe.

VIII.5.1.1 L'organisation de dataset et le format YOLO :

format Ultralytics YOLO est un format de configuration du jeu de données qui te permet de définir le répertoire racine du jeu de données, les chemins relatifs vers les répertoires d'images d'entraînement/de validation/de test ou les chemins relatifs vers les répertoires d'images d'entraînement/de validation/de test. *.txt des fichiers contenant les chemins d'accès aux images, et un dictionnaire de noms de classes. Voici un exemple :

```
path: '/content/drive/MyDrive/PFE/Dataset_Bouisthami'
train: ../train/images
val: ../val/images
test: ../test/images

nc: 1
names: ['Date_fruit']
```

Figure 21 : l'organisation de dataset dans le fichier data.yaml.

Les étiquettes de ce format doivent être exportées au format YOLO avec une *.txt par image. S'il n'y a pas d'objets dans une image, aucune *.txt est nécessaire. Le *.txt Le fichier doit être formaté avec une ligne par objet en class x_center y_center width height format. Les coordonnées de la boîte doivent être en xywh normalisé (de 0 à 1). Si tes cases sont en pixels, tu dois diviser x_center et width par la largeur de l'image, et y_center et height par la hauteur de l'image. Les numéros de classe doivent être indexés à zéro (commencer par 0).



Figure 22 : Une image annoter avec Roboflow.

Le fichier d'étiquettes correspondant à l'image ci-dessus contient 2 personnes (classe 0) et une égalité (classe 27) :



Figure 23 : Annotation d'image précédente.

Chaque image de votre ensemble de données doit avoir un fichier .txt correspondant avec tous les objets de l'image avec un [id_class x0 y0 x1 y1] normalisé.

VIII.5.1.2 Comment entraîner YOLOv8 sur votre ensemble de données personnalisé :

- **Importer et configurer YOLOv8 :**

La première étape consiste à importer la bibliothèque nécessaire et à charger un modèle YOLOv8 pré-entraîné. Nous pouvons choisir parmi plusieurs versions du modèle (par exemple, yolov8n.pt, yolov8s.pt, yolov8m.pt, etc.), où n, s, m, etc., désignent respectivement les versions nano, small, medium, etc.

Tableau 2 : Les versions du modèle YOLOv8 pour la détection.

Model	size (pixels)	mAP ^{val} ₅₀₋₉₅	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

Voici comment on peut importer le modèle YOLOv8 :

```
from ultralytics import YOLO
model = YOLO("yolov8n.pt")
```

Figure 24 : Code d'importation du modèle yolov8.

- **Configurer l'entraînement :**

Ensuite, il faut configurer les paramètres d'entraînement. Cela inclut des options telles que le chemin vers le fichier data.yaml, le nombre d'époques, la taille du batch, la taille des images, etc.

Voici un script Python détaillé pour configurer et démarrer l'entraînement :

```
result = model.train(data=os.path.join(ROOT_DIR, "data_2.yaml"), epochs=30)
```

Figure 25 : Code d'entraînement du modèle yolov8.

Après L'entrainement, un dossier runs créé par défaut par les scripts d'entraînement YOLO pour stocker les résultats de l'entraînement, y compris les modèles sauvegardés, les logs, et les visualisations. Ce dossier contient un autre dossier nommé « weights », Ce sous-dossier contient les poids du modèle sauvegardés au cours de l'entraînement. Typiquement, il contient deux fichiers principaux :

- **best.pt** : Les poids du modèle correspondant à la meilleure performance sur l'ensemble de validation.
- **last.pt** : Les poids du modèle à la fin de la dernière époque d'entraînement. On peut utiliser ce fichier comme un modèle pour continuer l'entraînement.

VIII.5.2 YOLOv8 pour la Classification d'Images :

Classification d'Images est Assigner une classe unique à une image entière.

VIII.5.2.1 L'organisation de dataset :

Pour Ultralytics YOLO pour les tâches de classification, l'ensemble de données doit être organisé dans une structure spécifique de split-directoire sous la forme d'un tableau. Afin de faciliter la formation, les tests et les processus de validation facultatifs. Cette structure comprend des répertoires distincts pour la formation (train) et les tests (test), avec un répertoire optionnel pour la validation (val).

Chacun de ces répertoires doit contenir un sous-répertoire pour chaque classe de l'ensemble de données. Les sous-répertoires portent le nom de la classe correspondante et contiennent toutes les images de cette classe. Veille à ce que chaque fichier image porte un nom unique et soit stocké dans un format commun tel que JPEG ou PNG.

- **Répertoire d'Entraînement (train/)** : Ce dossier contient les images utilisées pour former le modèle.
- **Répertoire de Validation (val/)** : Ce dossier contient les images utilisées pour valider le modèle pendant la formation. La validation permet d'ajuster les hyperparamètres et d'évaluer la performance du modèle sur des données inédites, aidant ainsi à prévenir le surapprentissage.
- **Répertoire de Test (test/)** : Ce dossier contient les images utilisées pour évaluer la performance finale du modèle après la formation. Les données de test

sont complètement séparées de celles d'entraînement et de validation pour fournir une évaluation objective de la performance du modèle.

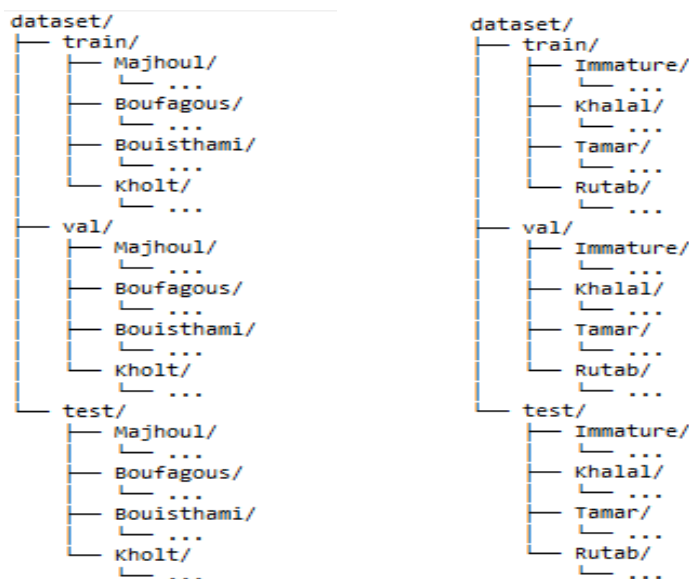


Figure 26 : l'organisation de dataset pour la classification selon le type et selon la maturité.

YOLOv8 offre une gamme de modèles pour la classification d'images, chacun conçu pour répondre à des besoins spécifiques en termes de ressources de calcul et de performance. Le tableau suivant présente quelque version :

Tableau 3 : Les versions du modèle YOLOv8 pour la classification.

Modèle	taille (pixels)	acc top1	acc top5	Vitesse CPU ONNX (ms)	Vitesse A100 TensorRT (ms)	params (M)	FLOPs (B) à 640
YOLOv8n-cls	224	69.0	88.3	12.9	0.31	2.7	4.3
YOLOv8s-cls	224	73.8	91.7	23.4	0.35	6.4	13.5
YOLOv8m-cls	224	76.8	93.5	85.4	0.62	17.0	42.7
YOLOv8l-cls	224	76.8	93.5	163.0	0.87	37.5	99.7
YOLOv8x-cls	224	79.0	94.6	232.0	1.01	57.4	154.8

IX. Conclusion :

Dans ce chapitre, nous avons défini les concepts clés de l'intelligence artificielle, du machine learning et du deep learning, en nous focalisant sur les réseaux de neurones convolutifs. Nous avons également introduit la vision par ordinateur et exploré la méthode YOLO, en particulier YOLOv8, pour la détection et la classification des objets. De plus, nous avons discuté de l'importance de l'organisation des datasets pour ces tâches. Dans le prochain chapitre, nous analyserons les résultats obtenus grâce à l'application de ces techniques sur notre projet de détection et de classification.

Chapitre 3 : Description de dataset

I. Introduction :

Dans ce chapitre, nous fournirons une description détaillée du dataset utilisé dans notre projet, y compris son origine, le nombre d'images par classe de classification, et les diverses catégories basées sur la variété et la maturité des dattes. Nous présenterons également les outils et technologies de développement employés pour mener à bien ce projet, tels que les langages de programmation et les logiciels de développement utilisés.

La détection automatique des fruits de dattes représente une avancée majeure dans l'optimisation des processus de récolte grâce à la vision par ordinateur et à l'apprentissage automatique. Le jeu de données intitulé "Date Fruit Detection Dataset for Automatic Harvesting" offre une base de données essentielle pour l'entraînement et l'évaluation des systèmes de détection des fruits de dattes, permettant d'automatiser la récolte, de réduire les coûts de main-d'œuvre et d'augmenter la productivité.

II. La description des données :

Le jeu de données contient 9092 images de grappes de dattes capturées à diverses échelles et angles entre juin et septembre 2022. Ces images ont été prises dans deux vergers au Maroc avec un appareil photo Canon et un smartphone Samsung Ultra. Les images sont annotées pour inclure les étapes de maturité des dattes (immature, Khalal,

Rutab, et Tamar) et varient selon les conditions d'éclairage, la couverture par des sacs, et l'angle de vue.

II.1 Conception Expérimentale, Matériaux et Méthodes

Les données ont été collectées dans deux vergers situés au Maroc, utilisant une caméra Canon et un smartphone Samsung Ultra. Les images sont annotées pour détecter et localiser les fruits de dattes en utilisant le logiciel Roboflow, avec des fichiers de sortie au format YOLO pour la détection d'objets.

II.1.1 Caractéristiques principales :

- **Types de données** : Images (JPG), vidéos (MP4), fichiers texte (TXT).
- **Variétés de dattes** : Majhoul, Boufaguos, Kholt, et Bouisthami.
- **Étapes de maturité** : Immature, Khalal, Rutab, Tamar.
- **Annotations** : Détection des fruits, classification des variétés, et classification des étapes de maturité.
- Le jeu de données est publié gratuitement sur ZENODO et vise à améliorer la recherche en agriculture robotisée en fournissant des données variées et annotées nécessaires pour développer des systèmes de vision machine robustes pour la récolte automatique des dattes.

II.1.2 Valeur des Données :

- **Modèles de détection d'objets** : Entraîner des modèles pour reconnaître et localiser les fruits de dattes dans un verger, permettant aux robots de naviguer et d'identifier les fruits prêts à être récoltés

- **Robotique en agriculture** : Aider à la classification des fruits de dattes à différents stades de maturité, pour des tâches telles que la détection, la segmentation, la récolte automatisée, l'analyse de maturité, le contrôle de qualité, et l'estimation du rendement.
- **Estimation du rendement** : Utiliser des algorithmes d'apprentissage automatique pour analyser le jeu de données et créer des modèles capables d'estimer avec précision le rendement de la récolte à venir, aidant ainsi les agriculteurs à prendre des décisions éclairées.

II.1.3 Description Détaillée des Données

Le jeu de données couvre toutes les étapes de la récolte, y compris la détection des dattes, la classification, l'analyse de maturité, et les décisions de récolte. Il comprend des images prises sous différents angles et conditions, telles que l'éclairage naturel, les dattes couvertes par des sacs, et les feuilles de palmier. Les stades de maturité des dattes sont classifiés en quatre catégories : Immature, Khalal, Rutab, et Tamar.

- **Étape Immature** : Connue sous le nom de Kimri, caractérisée par des dattes vertes, dures, et en croissance rapide.
- **Étape Khalal** : Les dattes changent de couleur, atteignent leur taille et poids maximums, prêtes pour une première récolte.
- **Étape Rutab** : Les dattes deviennent brunes ou noires, deviennent molles, idéales pour le stockage à froid.
- **Étape Tamar** : Dernière phase, avec des dattes brunes ou noires, texture douce et légèrement collante, conservation longue durée.

• Statistiques Générales

- **Total des images : 9092**

- **Total des grappes de dattes : 128**
- **Total des palmiers : 28**

Tableau 4 : Répartition par Stade de Maturité.

Maturité	Boufagous	Majhoul	Bouisthami	Kholt
Immature	913	304	95	140
Khalal	563	573	137	414
Rutab	1675	1345	46	649
Tamar	1174	423	82	559

Tableau 5 : Utilisation du Pourcentage de Maturité pour la Récolte.

Variété	Immature	Khalal	Rutab	Tamar
Boufagous	Non récoltée	Non récoltée	Récoltée (>30%)	Récoltée
Majhoul	Non récoltée	Non récoltée	Récoltée (>50%)	Récoltée
Bouisthami	Non récoltée	Non récoltée	Récoltée (>70%)	Récoltée
Kholt	Non récoltée	Non récoltée	Récoltée (>60%)	Récoltée

Tableau 6 : Nombre d'Images par Variété de Dattes

Variété de dattes	Nombre d'images	Nombre de grappes	Nombre de palmiers
Boufagous	4325	50	10
Majhoul	2645	58	14
Bouisthami	360	14	2

Kholt	1762	6	2
-------	------	---	---

II.2 Limitations :

Le jeu de données inclut quatre variétés de dattes, mais ne couvre pas l'ensemble des variétés existantes, ce qui peut limiter l'adaptabilité du modèle à d'autres types de dattes. De plus, il ne prend pas en compte les variations saisonnières dans l'apparence des dattes.

II.2.1 Déclaration d'Éthique

La production des données n'a impliqué aucun sujet humain, expérimentation animale, ni données provenant de plateformes de médias sociaux. Les auteurs ont respecté les exigences éthiques pour la publication dans Data in Brief.

II.2.2 Disponibilité des Données

Le jeu de données est disponible sur ZENODO sous le lien suivant : ZENODO.

III. Les outils utilisés :

Il est nécessaire de présenter les outils informatiques utilisés avant d'entamer la présentation des différentes parties qui constituent notre système de détection d'ilots.



III.1 Langage de programmation Python : python™

Python est un langage programmation open source le plus employé par les informaticiens.

Ce langage s'est propulsé en tête de la gestion d'infrastructure, d'analyse de données ou dans le domaine du développement de logiciels.

Malgré q'il ne soit pas le plus puissant du point de vue calcul et rapidité d'exécution, mais dans le domaine AI est le plus utilisé car sa syntaxe est facile et pratique, il permet d'y progresser très certainement plus aisément que dans d'autres langages.

Enfin, concernant le domaine de (DL) Python se distingue tout particulièrement en offrant une pléthore de bibliothèques de très grande qualité, couvrant tous les types d'apprentissages disponibles sur le marché.

III.2 Google colab :



Google Colab ou Colaboratory est un service Cloud, offert par Google (gratuit), basé sur Jupiter Notebook et destiné à l'entraînement et à la recherche dans l'apprentissage automatique. Cette plateforme permet d'importer un ensemble de données d'images, d'entraîner un classificateur d'images sur cet ensemble et d'évaluer le modèle de Deep Learning directement dans le Cloud.

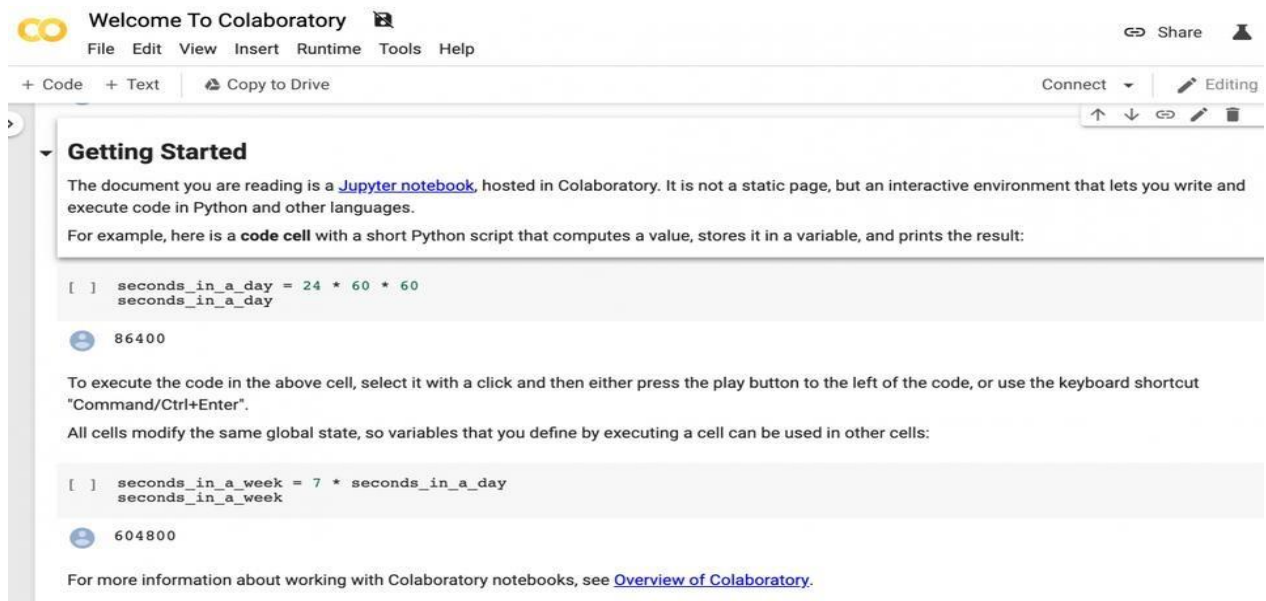


Figure 27 : Interface de Google Colab.

On a utilisé Google Colab, car il présente les avantages suivants :

- Aucune configuration requise : On n'a pas besoin d'installer quoi que ce soit sur notre ordinateur à l'exception d'un navigateur (Google chrome).
- Accès gratuit aux GPU : Les Notebooks Colab exécutent les codes sur les serveurs Cloud de Google, nous avons donc à notre disposition toute la puissance du matériel Google.

III.3 Google Drive : Google Drive

Google Drive est un service stockage et de partage de fichiers en ligne lancé par Google en avril 2012., il permet aux utilisateurs de stocker, partager, modifier et visualiser différents types de fichiers, et de les synchroniser à distance avec des terminaux fixes (PC, Mac) ou mobiles.

Google Drive est utilisé car :

- Il Permet de réserver un espace de stockage qui communique avec les autres services de Google (Google Docs, Google Colab).
- Il permet de Transfer des données entre Colab et Drive et de les synchroniser plus vite qu'avec une machine locale (PC, téléphone).
- Il permet une sauvegarde rapide et automatique de l'état d'avancement dans le Drive.

IV. Conclusion

Dans ce chapitre, nous avons décrit en détail le dataset utilisé pour la détection et la classification des dattes, en précisant son origine, le nombre d'images par classe, et les critères de classification. Nous avons également présenté les outils et technologies de développement employés dans notre projet. Dans le chapitre suivant, nous analyserons les résultats obtenus grâce à l'application de ces techniques et outils.

Chapitre 4 : L'interface web

I. Introduction :

Dans ce chapitre, nous présenterons l'interface web développée pour notre projet. Nous illustrerons également des exemples des résultats obtenus avec notre système de détection et de classification des dattes, en soulignant la précision et l'efficacité de l'application.

II. Les résultats obtenus avec YOLOv8 :

II.1 La courbe précision-rappel :

- La courbe précision-rappel est un graphique utilisé pour évaluer la performance d'un modèle de classification. Elle trace la relation entre la précision (precision) et le rappel (recall) pour différents seuils de classification.

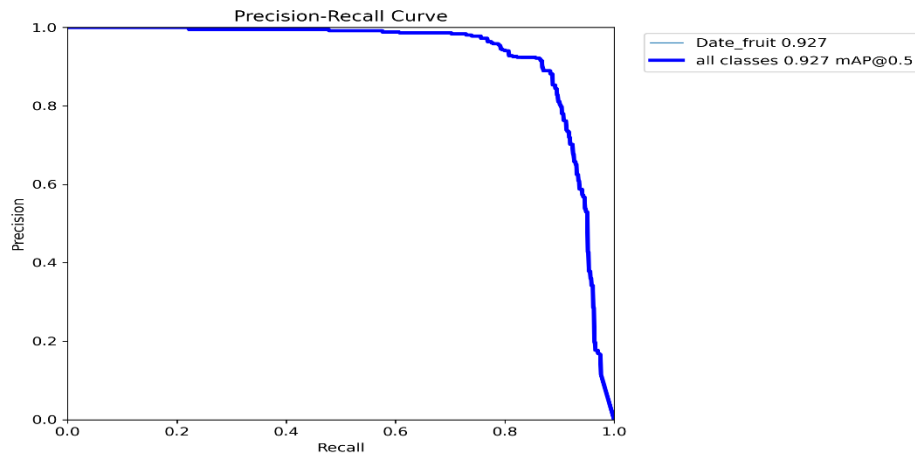


Figure 28 : La courbe précision-rappel.

- Axes du graphique :
 - Axe des abscisses (X) : Rappel (Recall)

Le rappel est la capacité du modèle à identifier toutes les instances pertinentes dans un jeu de données. Il est calculé comme le nombre de vrais positifs divisé par la somme des vrais positifs et des faux négatifs.

- Axe des ordonnées (Y) : Précision (Precision)

La précision est la capacité du modèle à ne pas étiqueter comme positif un échantillon qui est réellement négatif. Elle est calculée comme le nombre de vrais positifs divisé par la somme des vrais positifs et des faux positifs.

II.2 La matrice de confusion normalisée :

- La matrice de confusion normalisée évalue la performance d'un modèle de classification en montrant la répartition des instances des classes réelles dans les classes prédites.

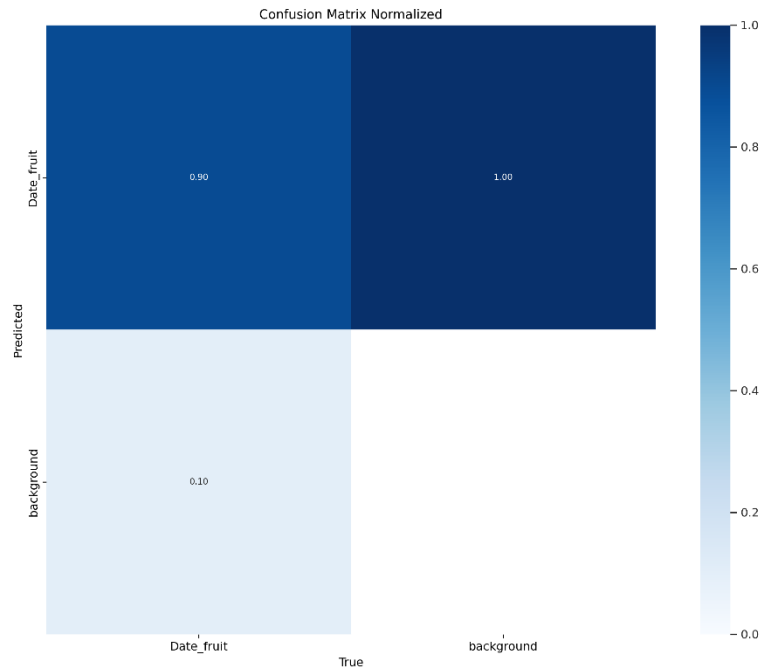


Figure 29 : La matrice de confusion normalisée.

- Axes :
 - Horizontal (True) : Classes réelles
 - Vertical (Predicted) : Classes prédites
- Interprétation des valeurs :
 - Les valeurs vont de 0 à 1 (0 à 100%).
 - Date_fruit (Vraie classe) :
 - 90% correctement prédites comme "Date_fruit".
 - 10% incorrectement prédites comme "background".
 - Background (Vraie classe) :
 - 0% incorrectement prédites comme "Date_fruit".
 - 100% correctement prédites comme "background".
- Couleurs :
 - Bleu clair : Valeurs faibles
 - Bleu foncé : Valeurs élevées

II.3 La matrice de confusion normalisée :

- La matrice de confusion normalisée évalue la performance d'un modèle de classification en montrant la répartition des instances des classes réelles dans les classes prédites.

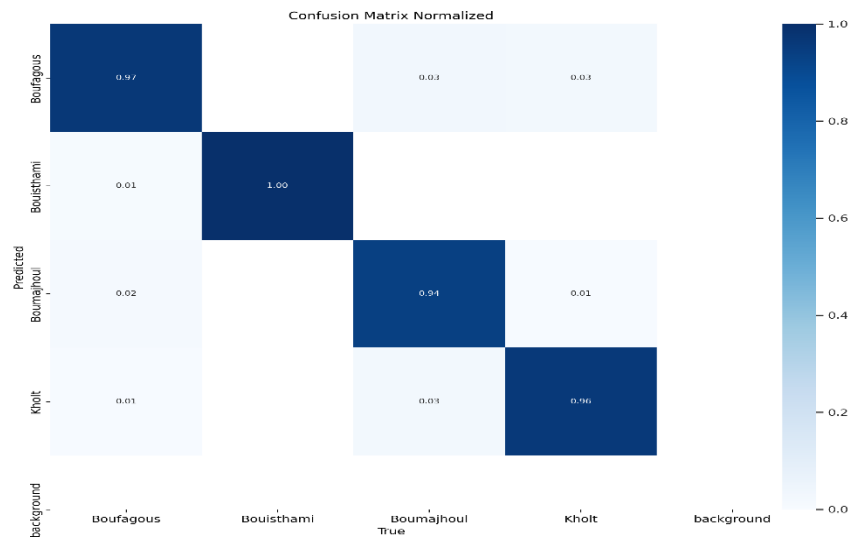


Figure 30 : La matrice de confusion normalisée

- Axes :
 - Horizontal (True) : Classes réelles
 - Vertical (Predicted) : Classes prédites
- Interprétation des valeurs :

Les valeurs sont normalisées et vont de 0 à 1 (0 à 100%).

- Analyse des résultats :
 - Boufagous (Vraie classe) :
 - 97% correctement prédites comme "Boufagous".
 - 3% incorrectement prédites comme "Boumajhoul" et "background".
 - Bouishtami (Vraie classe) :
 - 100% correctement prédites comme "Bouishtami".
 - Boumajhoul (Vraie classe) :
 - 94% correctement prédites comme "Boumajhoul".
 - 4% incorrectement prédites comme "Boufagous".
 - 1% incorrectement prédites comme "Kholt".
 - Kholt (Vraie classe) :
 - 96% correctement prédites comme "Kholt".
 - 3% incorrectement prédites comme "Boumajhoul".
 - 1% incorrectement prédites comme "background".
- Couleurs :
 - Bleu clair : Valeurs faibles
 - Bleu foncé : Valeurs élevées

II.4 Des graphiques de performance du modèle :

- Ces graphiques présentent les performances du modèle de classification au cours de l'entraînement, en termes de perte (loss) et d'exactitude (accuracy) pour l'ensemble d'entraînement et de validation.

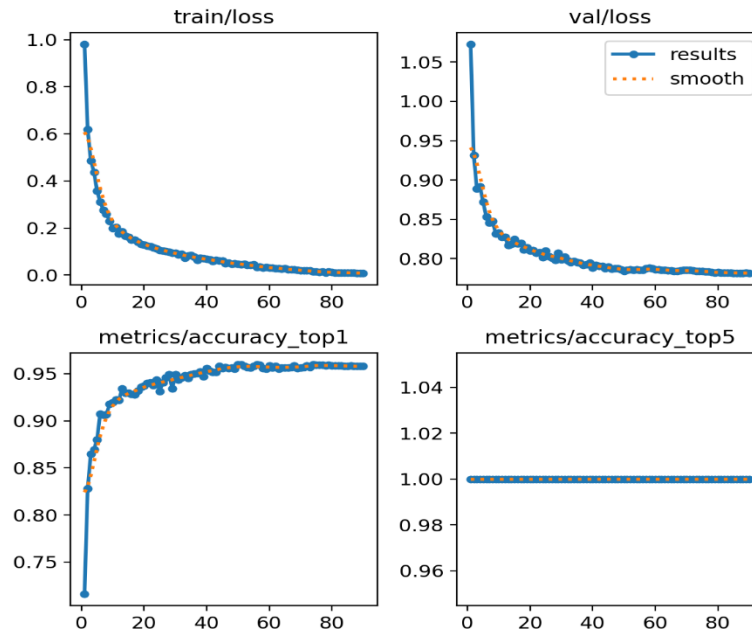


Figure 31 : les performances du modèle de classification au cours de l'entraînement

- Graphique de perte (Loss) :
 - train/loss (Perte d'entraînement) :
 - La courbe montre la diminution de la perte au fil des époques.
 - La perte commence à une valeur élevée et diminue progressivement, atteignant une valeur proche de zéro, ce qui indique que le modèle apprend correctement.
 - val/loss (Perte de validation) :
 - La courbe montre la diminution de la perte de validation.
 - La perte de validation suit une tendance similaire à la perte d'entraînement, ce qui suggère que le modèle généralise bien sans surentraînement.
- Graphique d'exactitude (Accuracy) :
 - metrics/accuracy_top1 (Exactitude de top 1) :
 - La courbe montre l'augmentation de l'exactitude de classification sur l'ensemble d'entraînement.
 - L'exactitude commence autour de 75% et atteint environ 95%, indiquant une amélioration significative de la performance du modèle.

- metrics/accuracy_top5 (Exactitude de top 5):
 - La courbe est constante à 1.00, ce qui signifie que pour chaque exemple, la bonne classe est toujours parmi les cinq meilleures prédictions du modèle.
 - Cela indique une très bonne performance du modèle.

III. L'interface graphique :

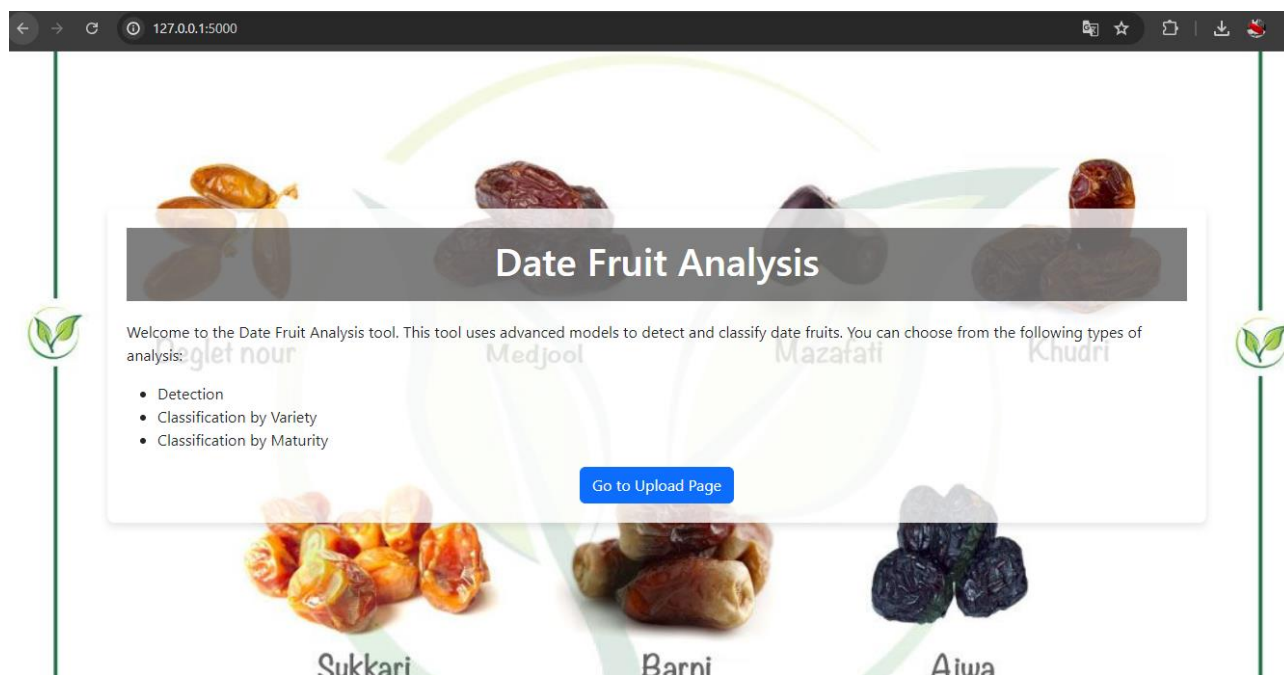


Figure 32 : page d'accueil

Un clic sur le bouton « Go to Upload page » affiche une deuxième fenêtre pour sélectionner une image.

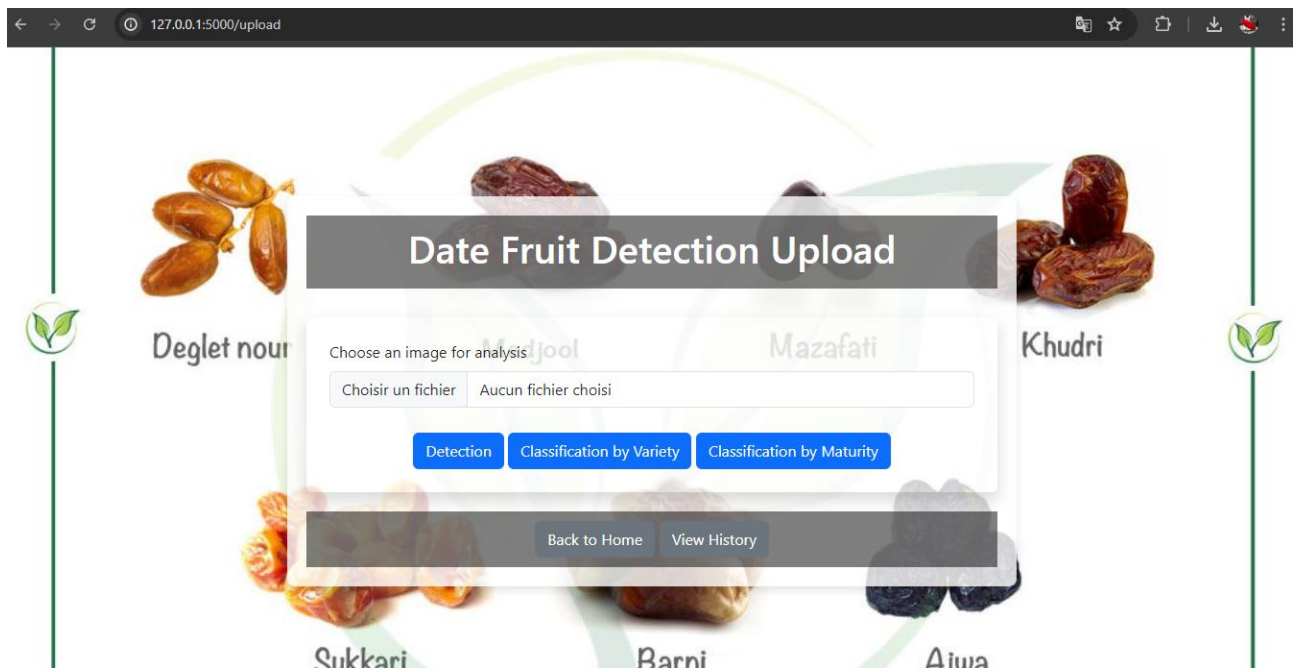


Figure 33 : d'importation des images.

➤ Options d'analyse :

Choisir une image à détecter :

- Cliquer sur le bouton "Choisir un fichier".
- Une fenêtre de sélection de fichier s'ouvre, permettant à l'utilisateur de sélectionner une image à partir de son ordinateur.

Trois boutons principaux sont disponibles pour l'utilisateur :

- **Détection** : Cette option permet d'identifier et de détecter les dattes présentes dans une image.
- **Classification par variété** : Cette option classe les dattes en fonction de leur variété, en utilisant des caractéristiques visuelles spécifiques.
- **Classification par maturité** : Cette option classe les dattes en fonction de leur stade de maturité, déterminé par leur apparence.

Le système téléchargera l'image et procédera à l'analyse pour analyser les fruits de dattes présents dans l'image.

Un clic sur l'un des boutons affiche la fenêtre des résultats.

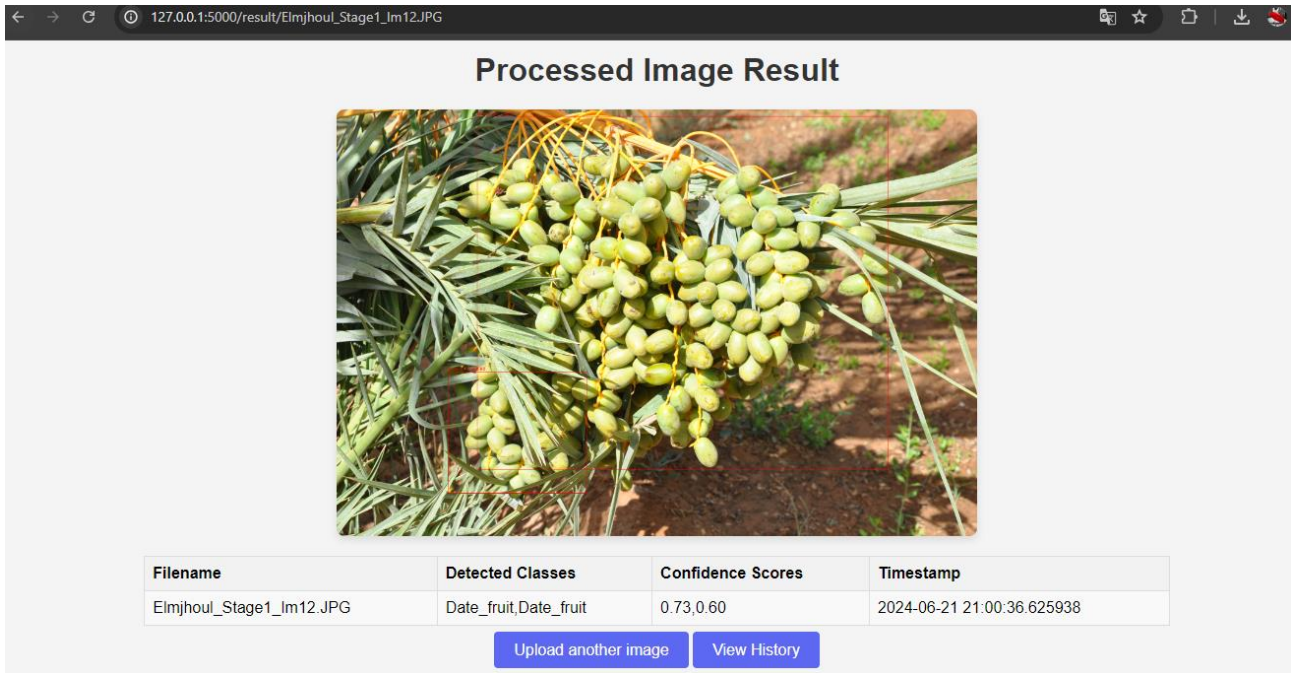


Figure 34 : page des résultats.

✓ Affichage de l'image traitée :

La page montre l'image de dattes traitée.

- Les dattes détectées sont encadrées : Des boîtes de délimitation (bounding boxes) entourent les dattes détectées dans l'image.
- Les classes détectées sont étiquetées : Les étiquettes indiquent les catégories des dattes détectées, par exemple "Date_fruit".

✓ Utilisation de la page :

- Visualiser les résultats : L'utilisateur peut examiner les résultats de l'analyse en regardant l'image traitée et les boîtes de délimitation autour des dattes détectées.
- Consulter l'historique : L'utilisateur peut vérifier les détails des analyses précédentes dans la page d'historique.
- Télécharger une nouvelle image : Si l'utilisateur souhaite analyser une autre image, il peut cliquer sur le bouton « Upload another image » pour retourner à l'écran de téléchargement.

Un clic sur le bouton « View History » affiche une fenêtre d'historique.

Cette page contient un tableau qui présent l'historique des analyses effectuées. Ce tableau contient les colonnes suivantes :

- Fichier « Filename » : Le nom du fichier image téléchargé.
- Classes détectées « Detected classes » : Les catégories de dattes détectées dans l'image.
- Scores de confiance « Confidence Scores » : Les scores de confiance associés à chaque détection, indiquant la fiabilité de la détection.
- Horodatage « Timestamp » : La date et l'heure à laquelle l'analyse a été effectuée.

Analysis History			
Filename	Detected Classes	Confidence Scores	Timestamp
Elmjhou1_Stage1_Im12.JPG	Date_fruit,Date_fruit	0.73,0.60	2024-06-21 21:00:37.961450
Elmjhou1_Stage1_Im12.JPG	Date_fruit,Date_fruit	0.73,0.60	2024-06-21 21:00:36.625938
DSC_2518.JPG	Khalal	0.91	2024-06-15 13:37:12.430034
DSC_2519.JPG	Boumajhoul	0.81	2024-06-15 13:36:37.899908
20220901_180324.jpg	Date_fruit,Date_fruit	0.72,0.67	2024-06-15 13:35:42.055455
DSC_3052.JPG	Date_fruit	0.79	2024-06-11 23:39:53.367043
20220725_181039.jpg	Boumajhoul	1.00	2024-06-11 23:38:22.409836
20220725_180911.jpg	Date_fruit	0.65	2024-06-11 23:37:30.601322
Elmjhou1_Stage2_Im54.JPG	Khalal	1.00	2024-06-11 23:33:00.885045
Elmjhou1_Stage2_Im24.JPG	Boufagous	0.78	2024-06-11 23:32:37.508888
20220925_123029.jpg	Date_fruit	0.86	2024-06-11 23:31:42.666578

Back to Home

Figure 35 : Historique des analyses des images précédentes.

Processed Image Result			
			
Filename	Detected Classes	Confidence Scores	Timestamp
Elmjhou1_Stage1_Im39.JPG	Boumajhoul	1.00	2024-06-21 21:35:11.978916

Upload another image View History

Figure 36 : Résultats de classification par variété

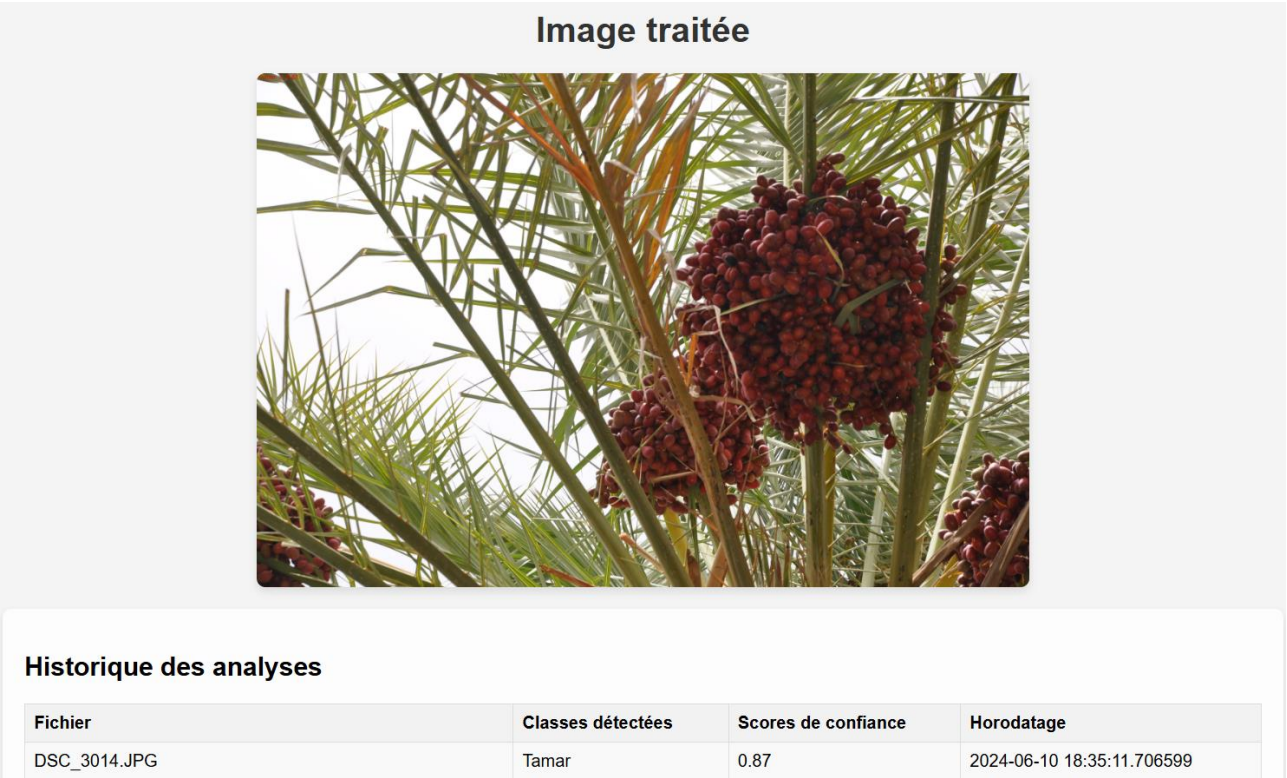


Figure 36 : Résultat de la classification par maturité

IV. Conclusion :

Dans ce chapitre, nous avons examiné l'interface web conçue pour notre projet et montré des exemples concrets des résultats obtenus. Nous avons démontré l'efficacité pratique de notre système de détection et de classification des dattes.

Conclusion générale

Au terme de notre cursus en diplôme, nous avons été chargés de réaliser un projet de fin d'études. Ce projet était axé sur la détection et la classification des objets, appliqué aux palmiers dattiers, ce qui nous a permis de découvrir une nouvelle discipline (le traitement d'image) et d'enrichir notre savoir ainsi que notre expérience.

Ce projet s'inscrit dans le cadre du diplôme en Génie informatique option Génie logiciel à l'université MOULAY SLIMANE, école supérieure de technologie de Khénifra.

Dans ce manuscrit, nous avons abordé la problématique de la détection d'objets et de la classification des dattes selon leur maturité et leur variété. Nous avons exploré diverses techniques de traitement d'image numérique, vectorielle et matricielle, afin de comprendre les défis associés à la gestion du bruit et à l'utilisation des histogrammes pour améliorer les images.

Nous avons étudié l'apprentissage automatique (machine learning) et l'apprentissage profond (deep learning), en particulier dans le contexte de l'intelligence artificielle. Le modèle YOLO (You Only Look Once), notamment la version Yolov8, a été implémenté pour la détection et la classification des objets.

YOLOv8 s'est révélé être un outil puissant pour la détection rapide et précise des dattes, grâce à son architecture de réseau neuronal convolutif (CNN) optimisée. Cette approche a permis de surmonter les défis liés à la variation de l'éclairage et aux différents angles de prise de vue des images.

Notre interface web développée permet de charger des images de dattes, de les traiter en temps réel et de fournir des résultats sur la maturité et la variété des dattes détectées. Cette interface se veut intuitive et accessible, facilitant ainsi son utilisation par des non-spécialistes.

Comme nous avons pu le constater à travers cette étude, l'application des différentes méthodes de détection et de classification a abouti à des résultats significatifs. Cependant, ces techniques nécessitent souvent des ajustements de paramètres spécifiques et une certaine connaissance préalable des données pour des résultats optimaux.

Pour conclure, notre travail peut être sujet à des extensions et des modifications. En effet, nous envisageons d'ajouter davantage de fonctionnalités, comme une base de données pour enregistrer les résultats et les utiliser dans des processus de reconnaissance plus avancés, ainsi que l'intégration d'autres algorithmes de détection pour enrichir notre application et démontrer encore plus l'utilité de ce domaine fascinant.

Bibliographie

- [1] **Vector graphics vs. Raster graphics:** <https://www.adobe.com/creativecloud/file-types/image/comparison/raster-vs-vector.html>
- [2] **Image vectorielle et image matricielle :** www.macapflag.com
- [3] **Introduction à la détection d'objets en Computer Vision [2024]:** www.innovatiana.com
- [4] **Date Fruit Classification for Robotic Harvesting in a Natural Environment Using Deep Learning:** <https://ieeexplore.ieee.org/document/8807111>
- [5] **Comment-entraîner-yolov8-dataset-personnalise#4-Le-format-YOLO :** www.picsellia.fr
- [6] **Ultralytics :** docs.ultralytics.com
- [7] **Wikipédia,** www.wikipédia.org
- [8] **M. ISMAIL IALAOUI El Mehdi, Cours de Master IAAD, Université MOULAY ISMAIL, Apprentissage profond – Réseaux de neurones convolutifs (CNN)**
- [9] **You Only Look Once (YOLO), Qu'est-ce que c'est ? :** www.datascientest.com
- [10] **H. D. Cheng, X.H. Jiang, Y. Sun & J. Wang, Color image segmentation : advances and prospects, Pattern Recognition, vol. 34, no. 9, 2001, pp. 22592281.**
http://www.mathworks.com/products/fuzzylogic/demos.html?file=/products/demos/shipping/fuzzy/fcmdemo_codepad.html
- [11] Academia.edu.com
- [12] **- Jean-Marie Beaulieu, Reconnaissance des formes : Classification et regroupement, département d'informatique, Université Laval, 2002.**
http://www.ift.ulaval.ca/Info_Courante/Vitrine/Syllabus_IFT64321.pdf